

Contents

1	声明	4
2	前言	5
2.1	代码转换复杂度	5
2.1.1	Java 到仓颉的转换挑战	5
2.2	J2CJ 的功能与局限	5
3	安装与设置	6
3.1	版本	6
3.2	系统要求	6
3.3	命令行工具安装	6
3.4	IntelliJ IDEA 插件安装	6
3.5	IntelliJ IDEA 插件配置	8
4	用法	10
4.1	IntelliJ IDEA 插件	10
4.1.1	配置插件	10
4.1.2	IntelliJ IDEA 插件使用说明	10
4.2	J2CJ 命令行	16
4.2.1	命令行选项	17
4.3	高级用法	18
4.3.1	增量转换	19
4.4	保留语义	20
4.4.1	Volatile 字段类型	20
4.4.2	函数类型	22
4.5	转换结果	23
5	映射	24
5.1	映射加载过程	24
5.2	添加自定义映射	24
5.3	映射语法	25
5.3.1	仓颉声明	25
5.3.1.1	扩展声明	26
5.3.2	Java 映射	27
5.3.2.1	类型映射	27
5.3.2.2	成员和构造函数映射	27
5.4	其他方法和构造函数映射注意事项	28
5.4.1	映射重载方法和构造函数	28
5.4.1.1	Java 签名	28
5.4.2	映射继承的方法	29
5.5	更多简单映射示例	29
6	注解支持	31
6.1	Java 标准注解类型	31

6.2	自定义注解类型	31
6.2.1	注解类型声明	31
6.2.2	使用注解	32
6.3	注解映射	32
7	J2CJ 不支持的 Java 特性	33
7.1	词法结构	33
7.1.1	注释	33
7.1.2	字面量	35
7.1.2.1	数值字面量	35
7.1.2.2	字符串字面量	35
7.2	类型、值和变量	35
7.2.1	引用类型	35
7.2.2	参数化类型	36
7.2.3	交集类型	36
7.3	转换和上下文	36
7.3.1	装箱转换	36
7.3.2	拆箱转换	36
7.4	名称	36
7.4.1	完全限定名称	37
7.4.2	范围	37
7.5	包	37
7.5.1	静态导入	37
7.6	类	37
7.6.1	类声明	37
7.6.1.1	类修饰符	37
7.6.1.2	内部类和封闭实例	37
7.6.2	字段声明	37
7.6.2.1	transient 字段	37
7.6.2.2	volatile 字段	37
7.6.3	方法声明	38
7.6.3.1	方法修饰符	38
7.6.4	实例初始化器	38
7.6.5	静态初始化器	38
7.6.6	枚举类型	38
7.6.7	记录类	38
7.7	接口	38
7.7.1	接口声明	38
7.7.2	字段（常量）声明	39
7.7.3	方法声明	40
7.7.4	注解类型	40
7.7.4.1	预定义注解类型	40
7.7.5	函数式接口	40
7.8	块和语句	40
7.8.1	标签语句	40
7.8.2	switch 语句	40

7.8.3	基础 for 语句	40
7.8.4	增强 for 语句	42
7.8.5	break 语句	42
7.8.6	continue 语句	42
7.8.7	synchronized 语句	42
7.8.8	try 语句	42
7.8.9	模式匹配	42
7.9	表达式	43
7.9.1	主要表达式	43
7.9.1.1	this	43
7.9.2	this 表达式	43
7.9.3	字段访问表达式	43
7.9.4	方法调用表达式	44
7.9.5	方法引用表达式	44
7.9.6	关系运算符	45
7.9.6.1	类型比较运算符 instanceof	45
7.9.7	lambda 表达式	45
7.10	其他	46
7.10.1	异常处理	46
7.10.2	元类型编程	46
8	标记	47
8.1	No type mapping: <CLASS_FQ_NAME>	47
8.2	Missing mapping for <CLASS_FQ_NAME> member/constructor: <MEMBER_NAME>	47
8.3	Missing method of mapped type <CLASS_FQ_NAME>: <MEMBER_NAME>	47
8.4	Mapping is present but failed	47
8.5	Unimplemented feature <FEATURE_NAME>	47
8.6	Unsupported <FEATURE_DESCRIPTION>	47
8.7	No type transform for: <EXPRESSION> <FROM_TYPE> -> <TO_TYPE>	48
8.8	Cannot infer type argument: <TYPE_PARAMETER>	48
8.9	Generic instantiation failure: <CALL>	48
8.10	Constraints not satisfied <FAILURES>: <CALL>	48
8.11	Expression is too deep: <EXPRESSION>	49
8.12	Constants in interfaces with bounded type parameters are unsupported: <INITIALIZER>	49
8.13	Unary operators usage with volatile fields is not supported: <EXPRESSION>	50
8.14	Compound assignment to volatile fields is not supported: <EXPRESSION>	50
8.15	Internal error	50

1 声明

1. 本工具仅供参考和学习使用，除非适用法律要求或有其他书面约定，本工具“按原样”提供，本工具权利人、开发者、提供者不提供任何明示或暗示的担保，不对使用该工具所导致的任何直接或间接损失或损害负责，包括但不限于利润损失、数据丢失、业务中断等。使用者需自行承担使用该工具所产生的任何风险、责任及/或任何损失。
2. 本工具产生的代码可能不完整、存在错误或缺陷，使用者需自行进行代码审查和测试，确保其适用性和正确性。
3. 本工具转化后的代码可能受到知识产权或其他合法权利的保护，使用者需遵守所有适用的法律法规，不得侵犯任何第三方权利。本工具所有者、开发者及提供者对使用者侵犯第三方权利的行为不承担连带责任、侵权赔偿责任等任何直接或间接责任。
4. 本工具所有者、开发者及提供者不对使用者基于使用本工具所开发的软件质量、可销售性、安全性或性能等做任何保证，使用者需自行承担相关风险和责任。
5. 本工具权利人有权对本声明的内容进行修改，使用者应定期查阅适用的最新版本。

2 前言

J2CJ 是一个旨在帮助开发者加速应用转化效率的半自动工具，在复杂场景下无法做到端到端全自动转化。

以下章节简单解释了不同语言间转化的技术挑战和限制。

2.1 代码转换复杂度

J2CJ 旨在进行源代码到目标代码的转换，能够具备以下特点：

- **可用**：假设原始程序无误，转换后的程序应在相同输入下产生相同输出。
- **快速**：转换后的程序应与原始程序的关键性能特性相当。
- **可读**：转换后的程序代码应具有良好的可读性和可维护性，所有注释应保留并恰当放置。

2.1.1 Java 到仓颉的转换挑战

Java 和仓颉都是用于通用场景的、面向对象的命令式编程语言，尽管初看上去它们有很多相似点，但仔细查阅会发现它们在核心层面存在部分差异，并且在诸多细节上存在大量差异。以下列举了几项关键点：

- 仓颉不具备 `null` 类型。因此无法直接将 Java 中的引用类型转换为仓颉引用类型，除非能确保该引用永不为 `null`，而后者需要相当复杂的程序流分析。
- 仓颉的字符串和数组不是对象，而在 Java 中它们是对象。此外，仓颉数组不支持协变。
- 仓颉不允许嵌套类型声明，例如，仓颉不支持内部类，这意味着 Java 中所有的内部类都需要通过显式声明和初始化外部实例引用来模拟，并需要使用更宽松的访问修饰符。

由于发展时间更长，Java SE 的标准库 API 相较仓颉更为丰富和庞杂，这也会为转换工作带来额外的复杂度，这也是 J2CJ 工具要重点解决的问题之一。

2.2 J2CJ 的功能与局限

J2CJ 的初衷并非是要全面替代那些精通 Java 和仓颉的专家，而是作为一种辅助工具，旨在协助 Java 库、应用程序以及框架转换为仓颉。在语言之间存在显著差异、无法自动转换的情况下，J2CJ 会生成一个 **标记**，让开发者来决定更有效的处理方式：是对转换生成的仓颉代码进行后处理或手动编辑，还是修改原始的 Java 源代码以便更好地映射到仓颉。更多详细信息，请参阅[高级用法](#)。

3 安装与设置

3.1 版本

J2CJ 版本号: 1.7.2

目标代码对应的仓颉版本号: 1.0.0

3.2 系统要求

硬件/操作系统: 支持编译和运行 17 控制台应用程序的任何系统。

软件: 17, e.g. OpenJDK 17, 可以从<https://jdk.java.net/archive/>或Eclipse Temurin下载。

3.3 命令行工具安装

1. 如果尚未安装 JDK 17, 请安装相应 JDK 版本。 > 注意 这并不是用于构建和运行 Java 项目的 JDK, 而是 J2CJ 翻译器本身运行所需的 JDK。这就是为什么需要指定特定版本的原因。 > 但如果你的项目目标也是使用相同版本的 JDK, 那么你可以继续使用现有的安装版本。
2. 将 J2CJ 压缩包 `j2cj-distrib-1.7.2-1.0.0.zip` 解压到所需的文件系统位置, 例如 `/opt/j2cj`。
3. 在终端中运行以下命令以验证安装:

```
/path/to/jdk/17/bin/java -jar /path/to/j2cj/j2cj.jar
```

例如:

```
/opt/openjdk-17.0.2/bin/java -jar /opt/j2cj/j2cj.jar
```

如果出现以下内容则说明 J2CJ 安装成功:

```
Loading built-in root.cjmap
Loading built-in stdlib.cjmap
Loading built-in mappings.cjmap
```

如果需要在 IDEA 中使用 J2CJ 插件, 请参考如下安装方式。

3.4 IntelliJ IDEA 插件安装

版本要求:

IntelliJ IDEA 2021.3 及以上 Android Studio Dolphin 2021.3.1 及以上

重要: 如果你已经安装了旧版本的插件, 请在插件更新过程结束时, 按照以下说明使 **IntelliJ IDEA/Android Studio** 缓存失效

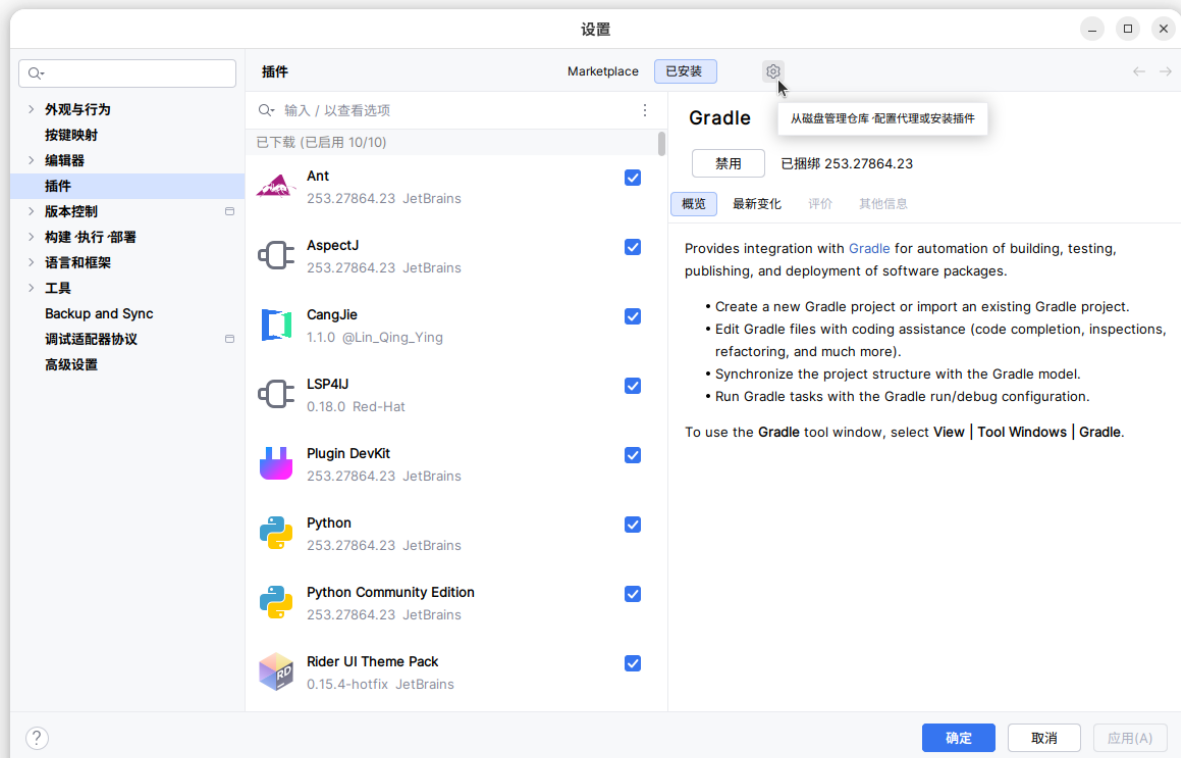
注意: 以下截图是在运行最新版本的 **IntelliJ IDEA** 截取的。在旧版本和 **Android Studio** 中, 相应的对话框可能看起来略有不同

IntelliJ IDEA 的 J2CJ 插件支持 Java 和 Android 项目。

要安装插件, 请按照以下步骤操作:

1. 在安装过程中解压 J2CJ 时所选择的目录中, 找到插件的 `.zip` 文件 (`j2cj-plugin-1.7.2.zip`)。

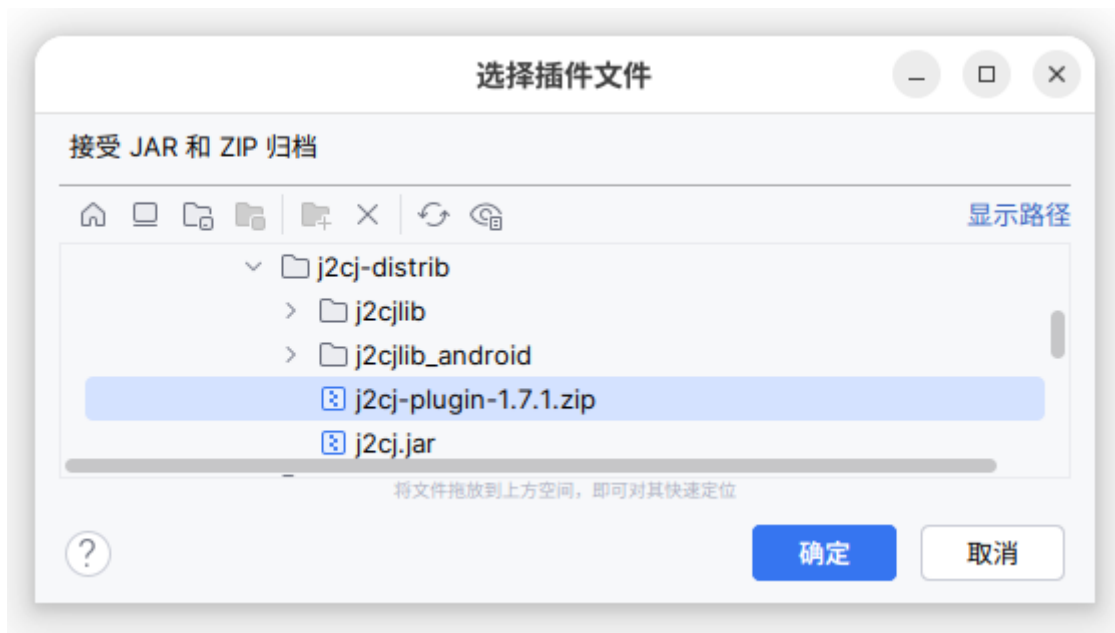
- 在 IDE 中，从主菜单中选择 文件 -> 设置，或者按下 **Ctrl+Alt+S** (PC) / **⌘Cmd**, (Mac)。
- 在“设置”对话框的左侧列中，选择“插件”。
- 点击窗口标题下方的齿轮图标：



- 选择“从磁盘安装插件”。



- 粘贴插件 **.zip** 文件的路径名，或在文件选择器中指向该文件，然后点击 **确定**。



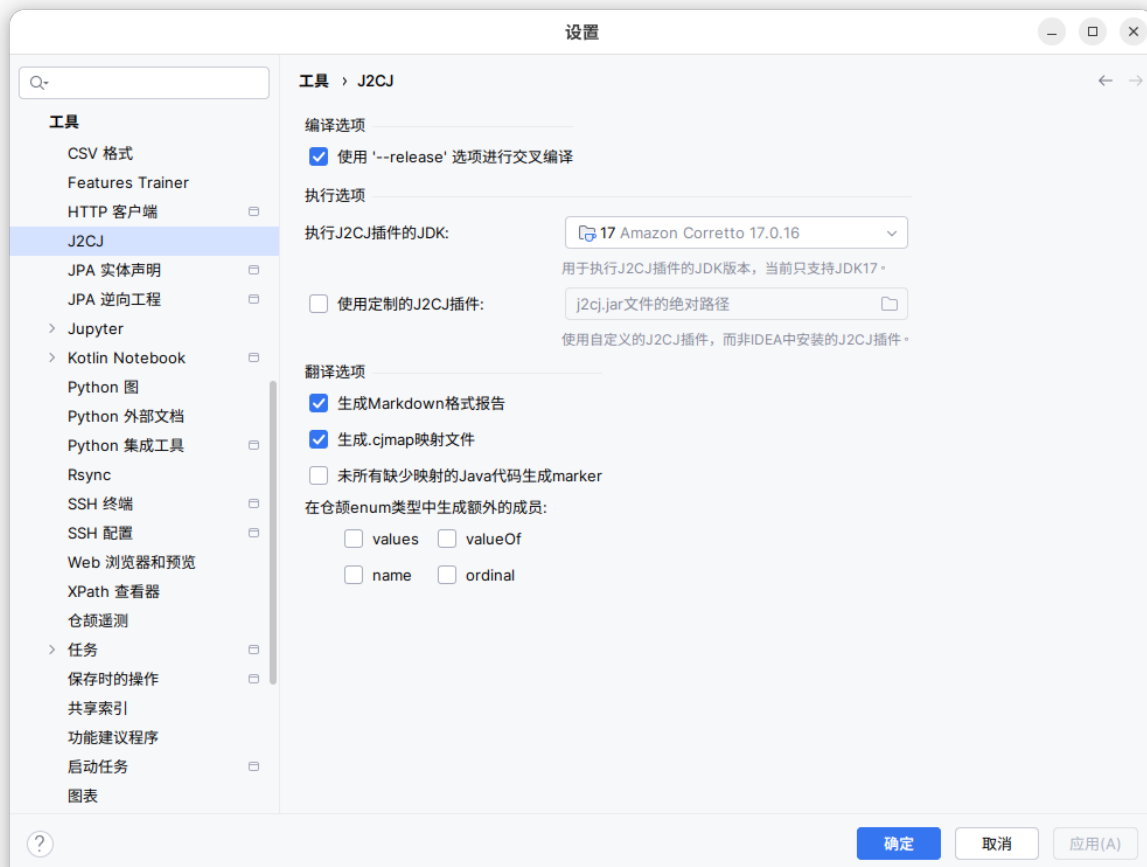
7. **重要:** 如果你已经替换了旧版本的插件，请立即失效 **IDE 缓存** 以确保正常运行：

- 在主菜单中，选择 **文件 / 无效缓存...**。
- 勾选前两个选项：**清除文件系统缓存和本地历史记录**，以及**清除版本控制系统日志缓存和索引**。
- 点击“失效并重启”。

3.5 IntelliJ IDEA 插件配置

要查看和/或更改 J2CJ 的 IntelliJ IDEA 插件设置：

1. 从主菜单中选择“文件”->“设置”，或者按 **Ctrl+Alt+S** 快捷键，设置弹出窗口将会打开。
2. 在左侧窗格中选择“工具”，然后在右侧窗格中点击“J2CJ”链接。或者，双击左侧窗格中的“工具”，然后在下拉菜单中选择“J2CJ”。J2CJ 插件设置选项卡将会显示：



3. 查看和/或更改设置，然后点击 **确定** 以确认所做的更改（如果有的话），或者点击 **取消** 以关闭 **设置** 窗口而不保存更改。

当前版本的插件支持以下基本设置：

- **执行 J2CJ 插件的 JDK** - 用于运行 J2CJ 的 JDK。JDK 的版本必须与 **系统要求** 中指定的版本一致，该版本也显示在此控件中，并在下方的消息中进行了标注。

注意 这并不是用于构建和运行 Java 项目的 JDK，而是一个专门用于运行 J2CJ 翻译器的 JDK。这就是为什么需要指定特定版本的原因。但如果你的项目所针对的 Java 版本与运行 J2CJ 所需的版本相同，那么你可以随意使用相同的 JDK。

通常情况下，您只有在 J2CJ 支持专家的指导下才会更改其他设置：

- **使用 ‘--release’ 选项进行交叉编译** - 通常勾选 以强制使用 Java 9 `javac` 编译器引入的 `--release` 选项。该插件设计为自动检测需要使用该选项的情况；只有在需要排查该启发式方法可能失败的情况下，才需要取消勾选此选项。
- **使用定制的 J2CJ 插件** - 粘贴一个独立的 J2CJ jar 文件的路径名，以替代插件中包含的 jar 文件，或者通过文件选择器指向该文件，然后点击 **确定**。
- **生成 Markdown 格式报告** - 通常勾选以强制生成 Markdown 格式的翻译报告文件。
- **生成 .cmap 映射文件** - 通常勾选以使 J2CJ 将翻译后的类的映射信息保存在输出目录中的 `.cmap` 文件中，以便后续在增量翻译场景中使用。
- **未所有缺少映射的 Java 代码生成 marker** - 通常不勾选；勾选此选项可能有助于开发自定义 **映射**，但翻译比率数据将会失真，因为 每个缺少映射的单个实体都会被包裹在一个 **标记** 中，从而降低整体的翻译比率。
- **在枚举 enum 类型中生成额外的成员** - 请参见 `--enum-functions` 命令行选项 的描述。

4 用法

使用 J2CJ 最简单和推荐的方法是通过 **IntelliJ IDEA 插件**。如果无法使用特定 IDE 或需要进行 **增量转换**，则可以使用 **J2CJ 命令行**。

4.1 IntelliJ IDEA 插件

重要提示：已安装旧版插件的用户，请确保更新完成后按以下步骤 **7** 描述清理 **IntelliJ IDEA** 缓存。

J2CJ 提供的 IntelliJ IDEA 插件可单独下载，支持 Java 和 Android 项目开发。

安装插件的步骤如下：

1. 下载插件 (**j2cj-plugin-X.Y.Z.zip**，其中 **X.Y.Z** 表示插件版本号)。
2. 进入 IDEA，在主菜单中选择 **文件 > 设置**，或者使用快捷键 **Ctrl+Alt+S**。
3. 选择 **插件**。
4. 单击窗口标题下方的齿轮图标。
5. 选择 **从磁盘安装插件**。
6. 复制粘贴 **.zip** 插件包的路径名或在文件选择器中选择该文件，然后单击 **确定**。
7. **重要提示：**若更新插件，请清理 IntelliJ IDEA 缓存，以确保软件正常运行。
 - 在主菜单中选择 **文件 > 使缓存失效...**。
 - 勾选 **清除文件系统缓存和本地历史记录** 以及 **清除 VCS 日志缓存和索引**。
 - 单击 **失效并重启**。

4.1.1 配置插件

要查看或更改插件设置，请参考以下操作：

1. 在主菜单中选择 **文件 -> 设置**，或按组合键 **Ctrl+Alt+S**，打开设置窗口。
2. 在窗格中，选择 **工具**，然后单击右窗格中的 **J2CJ** 链接。或者，双击窗格中的 **工具**，然后选择 **J2CJ** 列表。
3. 查看和/或更改设置，然后单击 **确定** 确认更改，或 **取消** 以关闭 **设置** 窗口而不保存更改。

当前的插件版本支持如下基础设置

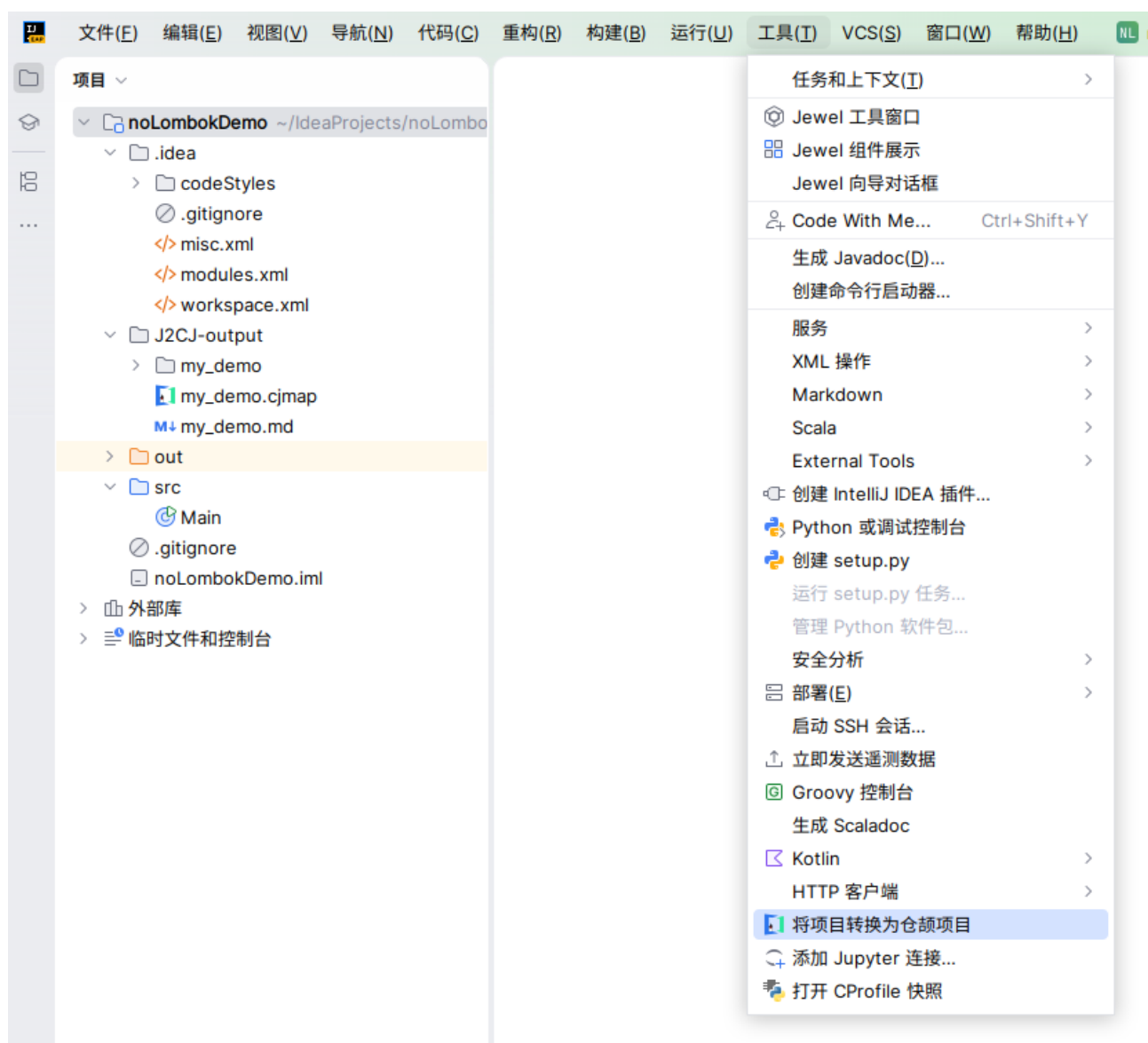
- **执行 J2CJ 插件的 JDK** - 开发者可以选择设置 JDK 版本来运行 J2CJ，请注意和 **系统要求** 里的 JDK 版本一致

注意：这里指定的不是构建运行开发者的 Java 工程的 JDK，而是一个单独用于运行 J2CJ 转换器的 JDK。但是如果碰巧两者可以用同一个 JDK 的话，那么可以不用单独设置。

4.1.2 IntelliJ IDEA 插件使用说明

1. 在 IDEA 中打开项目。

2. 从主菜单中选择 工具 -> 将项目转换为仓颉项目:

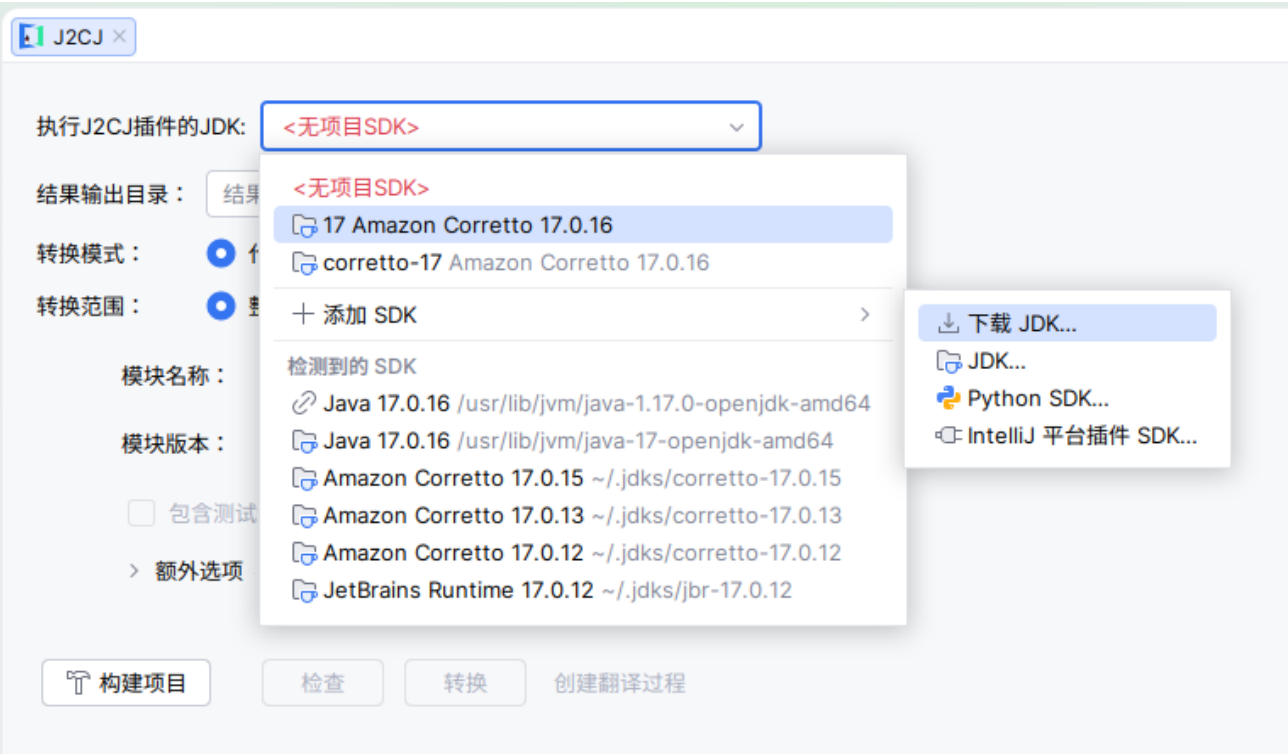


如下窗口会被打开:

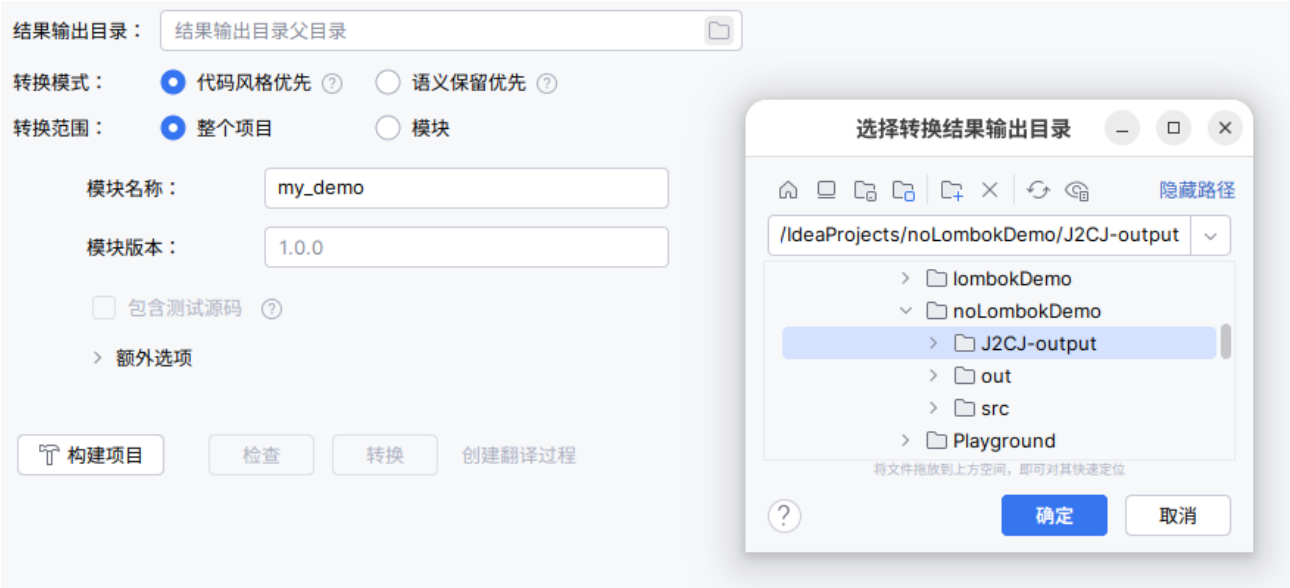


3. 如果显示 **执行 J2CJ 插件的 JDK** 下拉菜单，请选择一个 JDK 17 安装版本，以便 J2CJ 运行。

此下拉菜单仅在您首次转换项目时出现，并且只有在您在配置插件时（配置）未指定适合运行 J2CJ 的 JDK 安装路径的情况下才会显示。



4. 在结果输出目录字段中，输入 J2CJ 转换结果目标路径，或使用文件选择器进行选择。



5. 根据项目需要，选择合适的**转换模式**

- **代码风格优先** (默认的): 生成易读的代码，但转换后的代码可能需要开发者进一步手动修改
- **语义保留优先**: 生成的代码尽可能的保留了 **Java** 语义，但结果中可能包含比较冗余的代码。

参考[保留语义](#)获取更多的细节

6. 开发者可以选择在 **模块名称** 和 **模块版本** 字段中预设生成结果的模块名和版本号，默认情况下，**IDEA** 项目名称将用作仓颉模块的名称，其中任何不被 **CJPM** 允许的字符都会被移除。版本号设置为“**1.0.0**”。
7. 开发者可以选择是否 **包含测试源码**，如果是，那么 **J2CJ** 会将 **IntelliJ IDEA** 识别为测试用例的代码也一并转换。
8. 建议在使用 **J2CJ** 工具前先完成项目构建，选择 **构建项目** 完成构建
9. 单击 **检查** 检测插件是否能正常工作，如果检测到任何问题，请等待 **30-60** 秒，让 **IDE** 先完成内部索引，然后再进行 **check**。如果使用 **Gradle** 构建项目，请选择 **同步所有 Gradle 项目** 强制/加快进行索引构建
10. 检测成功后，单击**转换**。**J2CJ** 的输出将会显示在**运行**窗格中。

转换任务名称旁边将显示一个绿色勾号，表示任务成功完成：

注意：IntelliJ IDEA 插件会自动查找所有项目或模块根目录下的 `.cjmap` 文件并将其文件路径传给 J2CJ。详情见[添加自定义映射](#)。

在“附加选项”下会出现以下可选设置：

- 如果 J2CJ 的输出中包含少量“Expression is too deep”（表达式过深）的标记，可以增加“最大表达式深度”。警告：这可能会显著减慢翻译过程。重构 Java 代码是更安全的选择。有关该标记的详细描述，请参阅[标记](#)章节。

否则，如果将整个项目翻译成一个单独的仓颉模块是不可取的，则可以执行上述五个初步步骤，然后在“翻译范围”组中选择“模块”选项。其下方的整个项目设置将被替换为一个仓颉输出模块面板，该面板包含相同的字段和一个可编辑的 Java 模块列表，其中已预先填充了所有项目模块。此外，该面板下方还会出现一个“添加输出仓颉模块”按钮。

要将 J2CJ 的输出分配到多个仓颉模块上，请根据需要重复以下步骤：

- 点击“添加模块”按钮。另一个仓颉输出模块面板将出现在其上方，其中包含一个空的 Java 模块列表。
- 在刚刚添加的输出仓颉模块的“模块名称”和“模块版本”字段中，分别输入所需的名称和版本。（对于第一个输出模块，这些信息将从整个项目设置中复制而来；你也可以进行编辑。）

J2CJ ×

结果输出目录：

转换模式：☒ 代码风格优先 ? ☐ 语义保留优先 ?

转换范围：☐ 整个项目 ☒ 模块

仓颉模块

模块名称：

模块版本：

+ -

Java模块	状态
lombokDemo	OK
t1	OK
t2	OK

☒ 包含测试源码

> 额外选项

模块名称：

模块版本：

+ -

Java模块	状态
未包含Java模块 添加Java模块	

☐ 包含测试源码 ?

▼ 额外选项

最大表达式深度： ?

8. 对于每个你想要翻译成新添加的输出仓颉模块的 Java 模块：

- 如果该 Java 模块目前出现在其他输出仓颉模块面板中的 Java 模块列表中，请选择该模块的名称，然后点击列表上方的减号“-”按钮。插件将会将该模块从列表中移除。

- b. 点击新添加的输出仓颉模块上方的加号“+”按钮，选择你刚刚移除的模块，然后点击“确定”。插件将执行快速验证程序。

9. 可选地，如果现在输出仓颉模块的“包含测试源码”复选框已启用，你可以勾选它以同时转换 IDE 识别为测试的源代码文件。

一旦你对输入 Java 模块在输出仓颉模块之间的分布感到满意后，可以像往常一样调用翻译器：

10. 强烈建议在尝试翻译之前先进行项目的正常构建。如果你最近没有构建过该项目（或者从未构建过），请点击“构建项目”以启动构建过程，然后等待其完成。
11. 点击**检查**。插件将执行一次模拟运行，不会生成任何输出文件。如果检查过程中发现任何问题，请等待 30-60 秒，让 IDE 完成内部索引的更新，然后再次尝试检查。**注意：**如果您的项目使用 Gradle，您还可以使用 IDE 中的“同步所有 Gradle 项目”功能来强制/加速更新。
12. 一旦检查成功，点击**转换**按钮开始翻译过程。

J2CJ 消息将显示在 **运行** 面板中。如果在 **插件设置** 中勾选了 **生成 Markdown 格式报告** 选项，J2CJ 还会为每个输出的仓颉模块生成一份独立的翻译报告（有关翻译报告的更多信息，请参见上文）。

对于每个输出的仓颉模块，在 **额外选项** 下会出现以下设置：

- 如果 J2CJ 的输出中包含少量“Expression is too deep”（表达式过深）标记，可以调高 **最大表达式深度**。**警告：**这可能会显著减慢翻译过程。重构 Java 代码是更安全的选择。有关该标记的详细信息，请参阅 **标记** 章节中的描述。
- 如果你之后改变主意，可以通过点击 **删除模块** 按钮来删除指定的输出仓颉模块。你无法移除当前唯一的输出模块。

警告：无论该输出模块关联的 Java 模块列表是否为空，它都会被移除。如果该列表不为空，而你希望 J2CJ 继续翻译这些 Java 模块，你需要将这些 Java 模块添加到剩余的输出仓颉模块中。

最后，如果你还希望在翻译过程中完全跳过 Java/Android 项目中的某些模块，可以在 Java 模块列表中选中这些模块，然后点击列表上方的减号“-”按钮。

4.2 J2CJ 命令行

使用 J2CJ 命令行转化工程，要求工程可以通过单条 `javac` 命令成功编译。

重要提示：所有源代码必须在单个 `javac` 调用中编译，使用 Maven 或者 Gradle 等构建工具进行编译的项目，需要获得生成构建工具最终生成的 `javac` 构建命令。

获得 `javac` 命令行后，按如下方式进行修改：

- 保留所有 `javac` 参数不变。
- 仅将 `javac` 命令替换为 `java -ea -jar J2CJ-jar`，其中 `J2CJ-jar` 是在 J2CJ 安装中找到的 `j2cj.jar` 文件的路径名。

示例：

```
javac -cp lib.jar App.java
```

转换为：

```
java -ea -jar /opt/j2cj/j2cj.jar -cp lib.jar App.java
```

编辑后的命令行将调用 J2CJ，以尝试将 Java 源代码转换为仓颉。

以下是一个更复杂的 Bash 示例：


```
# Collect the pathnames of all Java source files located under ./src:
find ./src -type f -name "*.java" > sources.txt

# Do the same for their jar file dependencies collected under ./lib
# and join their pathnames together into a single classpath string:
export JARS="`(readarray -t ARRAY < <(find ./lib -type f -name "*.jar"); IFS=":");
echo "${ARRAY[*]}")`"

# Run J2CJ to convert all source files, placing the result in ./j2cj-output
# using "test" as the module name:
/path/to/JDK/17/bin/java -ea -Dmodule.name=test -jar /path/to/j2cj/j2cj.jar \
-d ./j2cj-output -classpath $JARS @sources.txt
```

注意：为了确保上述脚本正常运行，通常需要为依赖项提供映射。详情请参阅[添加自定义映射](#)和[增量转换](#)。

4.2.1 命令行选项

转换器能够识别 OpenJDK 17 支持的标准 `javac` 编译器的命令行选项，并且忽略任何不适用的选项，例如注解处理器的选项，但是 J2CJ 对注解支持非常有限，不支持编译期注入方式的注解处理。

若待转换的 Java 源代码的 JDK 版本早于 J2CJ 使用的 JDK 版本，则可以使用 `--system` 和 `--release` 选项控制编译输入/输出的 JDK 版本。

```
/path/to/JDK/17/bin/java -ea -Dmodule.name=legacy -jar /path/to/j2cj/j2cj.jar \
--system /path/to/JDK/11 --release 11 \
-d ./j2cj-output -classpath $JARS @sources.txt
```

如果原 Java 项目是模块化的，则可以使用 `--module-source-path` 和 `-m` 选项，而不是单独提供 `.java` 文件列表。

除了所有这些与 `javac` 兼容的选项之外，J2CJ 还提供了控制转换过程的选项。

重要提示：在当前版本中，J2CJ 特定的命令行选项必须作为 JVM 参数传递，即在 `java` 启动器的 `-jar` 选项之前通过 `-D` 选项传入参数：

```
java -ea -Dmodule.name=quux -jar /opt/j2cj/j2cj.jar -cp lib.jar App.java
```

以下是 J2CJ 额外支持的命令行选项：

`-Dgenerate.mapping.file=pathname`

将正在转换的代码映射保存到指定的 `pathname` 文件中。详细信息请参阅[增量转换](#)。

`-Dmodule.name=identifier`

在生成的仓颉包声明中使用 `identifier` 作为模块名。

每个仓颉源文件都属于特定的命名包，而该包又可能属于特定的模块。默认情况下，J2CJ 使用 Java 包名称最左侧的标识符作为模块名，所以将以下 Java 包声明：

```
package cu.mber.some;
```

转换为：

```
package cu.cu.mber.some
```

并存放于转化后的仓颉文件中。通过此选项可以指定一个特定的模块名来替代默认值。

`-Dmodule.version=string`

在生成的`cjpm.toml`文件中使用`string`作为模块版本号。默认值为`1.0.0`。

警告： J2CJ 不检查`string`是否为正确的版本号。

`-mode=semantic|codestyle`

- **codestyle**(默认模式)：生成更加可读可维护代码，如不默认生成 `Hashable` 和 `ToString` 方法，保留 `==`，`!=` 操作符，不替换成 `RefEqual` 函数，不生成冗余的 `open` 等。
- **semantic**：生成更加准确的代码，方便用户直接在项目中使用（但由于为了保证准确性，可能会生成冗余的 `HashTable`、`ToString` 等代码）

`--enum-functions=list`

在默认的“代码风格优先”模式（`-mode=codestyle`）下，强制生成某些 Java 枚举方法。在该模式下，J2CJ 在将 Java 枚举类型转换为 Cangjie 枚举时，通常不会合成 Java 枚举类型的某些内置方法的等价物，即 `values`、`valueOf`、`name` 和 `ordinal`。`list` 是一个逗号分隔的列表，包含一个或多个这些 Java 方法的名称。

示例：

```
--enum-functions=values,ordinal
```

在语义保留模式（`-mode=semantic`）下，此选项无效。所有四个函数始终存在于生成的类中。另请参阅枚举类型。

4.3 高级用法

对于存在语言特性差异、标准库差异的场景，J2CJ 无法保证这部分代码一定可以转换成功，但 J2CJ 会尽量尝试“近似正确”的转换，并使用 `<-- -->` 标记这部分代码，被标记的代码不符合仓颉语法，因此无法被正确编译，仅为开发者后续处理提供建议。

除了 J2CJ 自身的错误之外，转换失败还有两个主要原因：

1. 源代码使用了 J2CJ 暂时不支持的 Java 语言特性。有关当前不支持的特性列表及其实现预测，请参阅 J2CJ 不支持的 Java 特性。

示例：

在仓颉中没有 Java 实例初始化器的合适等效项，因此 J2CJ 会将以下 Java 类

```
class Foo{
    int x;
    {
        x = 1;
    }
}
```

转换成

```
open class Foo <: Hashable & ToString {
    init() { }

    var x: Int32 = 0

    <-- Unsupported initialization blocks: {
        x = 1i32
    } -->

    . . .
}
```

解决办法：修改 Java 源码，避免使用不兼容的功能。测试更改，确保 Java 代码正确性后，再通过 J2CJ 运行。

在上述示例中，可以很容易地将实例初始化器中的赋值替换为字段初始化器：

```
class Foo {
    int x = 1;
}
```

J2CJ 转化生成：

```
open class Foo <: Hashable & ToString {
    init() { }

    var x: Int32 = 1i32

    . . .
}
```

2. 源代码使用了 J2CJ 未知的 Java 平台 API 或第三方库。

解决办法：从以下任选一项：

- 为无法转化的 API 或库添加映射。
- 修改 Java 源代码，使用 J2CJ 中已有映射的 API 或库，然后再次尝试转化。

当然，在这两种情况下都可以对 J2CJ 生成的代码进行后置手动修改，以形成完整且正确的仓颉应用。

重要提示：对 Java 源代码进行任何变更时，建议通过测试的方式保证变更正确性，后使用 J2CJ 进行转化，以保证转化输入符合预期

4.3.1 增量转换

增量转换通常涉及将大型项目按模块分解或将转换过程分段进行。此外，当多个项目共享相同的库、模块或包时，也适合采用增量转换策略。

由于仓颉和 J2CJ 都不支持循环依赖，Java 代码如存在循环依赖，需重构这些依赖后使用 J2CJ 进行转换。

1. 将待转换的代码拆分成若干模块，确保这些模块之间不存在循环依赖；
2. 识别一组独立模块作为基础模块，这些模块不存在对其他模块的依赖；
3. 翻译这组“相对独立的”的基础模块并测试代码正确性；
4. 识别其他模块，其仅对之前翻译成功的基础模块存在依赖；
5. 翻译其他模块并测试代码正确性；
6. 重复步骤 4，直至完成全部模块翻译。

然而，为确保步骤 5 成功并避免重复转换，J2CJ 必须接受额外的输入，即这些部分的实际映射。如果没有这些映射，J2CJ 将无法正确转换依赖于已转换部分的任何 Java 代码。

为了简化增量转换流程，J2CJ 可以选择性地将正在转换的代码的映射保存到 `.cjmap` 文件，并作为其他模块/部分转化的输入。

可通过设置系统属性 `generate.mapping.file` 来指定 `.cjmap` 文件路径名，例如 `-Dgenerate.mapping.file = ". /common.cjmap"`。

注意：系统属性是作为 `java` 命令的参数传递，而不是作为 J2CJ 本身的参数。在转换一组依赖的 Java 源文件时，需将生成的 `.cjmap` 文件放入 `root.cjmap`。

示例:

转换常用模块:

```
java -ea -Dgenerate.mapping.file="./common.cjmap" -jar j2cj.jar \
  -d "./common" @common.lst
```

在转换其他模块时添加它们的映射:

```
#include "stdlib.cjmap"
#include "mappings.cjmap"

#include "custom.cjmap"
#include "common.cjmap"
```

4.4 保留语义

默认情况下, J2CJ 力求生成符合习惯的仓颉源代码, 即使这意味着可能会丢失或改变原始 Java 代码的某些语义。以 “codestyle” 模式生成的仓颉源代码通常 更小、更易读/维护, 但也可能包含更多的[标记](#), 并且需要更多的翻译后分析和编辑。

在保留语义模式 (`-mode=semantic`) 下, J2CJ 的翻译规则如下:

1. 所有 Java 类要么实现要么继承 `equals`、`hashCode` 和 `toString` 方法, 因为这些方法存在于 Java 类层次结构的根类 `java.lang.Object` 中。但这些方法的仓颉实现为 `Equatable<T>`、`Hashable` 和 `ToString` 接口。如果指定了 `-mode=semantic` 选项, J2CJ 会将这三个接口添加到所有转换后的类中, 并合成它们的实现。这使得转换后的仓颉代码明显变大, 但同时如果原始 Java 代码依赖于这些方法中的任何一个或全部的存在, 转换后的代码更不容易出错。
2. 应用于引用类型值的 Java 相等性测试运算符 `==` 和 `!=` 比较的是引用本身。J2CJ 默认按字面翻译它们。然而, 对于未实现 `Equatable<T>` 接口的类型, 这些仓颉运算符可能未定义。此外, 对于每个特定的引用类型, 它们可能被实现为引用比较、值比较、部分值比较、哈希值比较或其他方式。当 `-mode` 选项设置为 `semantic` 时, 如果 `a` 和 `b` 在原始代码中具有 Java 引用类型, J2CJ 会将 `a == b` 翻译为 `refEq(a, b)`。
3. J2CJ 通常会省略那些没有已知子类和子接口的类和接口上的 `open` 修饰符, 以及未重写/未重新定义的成员函数上的 `open` 修饰符, 即使原始 Java 类型和方法不是 `final`。在 `-mode=semantic` 下, 它会将每个仓颉接口、非抽象类和成员函数标记为 `open`, 除非原始 Java 类型或方法是 `final`。
4. 一个辅助库 `j2cjlib` 被用来提高翻译的语义准确性和完整性。特别是某些复杂的映射关系, 如果不使用这些函数, 就无法实现; 或者在使用默认的“代码风格优先”模式下, 其实现的准确性和/或效率会较低。[映射关系](#)
5. 函数类型不用于翻译函数式接口的使用, `lambda` 表达式总是被翻译成类。
6. 除了 `boolean` 和整数类型之外, 其他类型的 `volatile` 字段会被不同的方式翻译。详情请参阅下面的 [volatile 字段类型](#)。
7. J2CJ 在确定是否可以将基本的 `for` 语句转换为仓颉 `for-in` 循环时会进行更严格的检查。详情请参阅 [基础 for 语句](#)。

4.4.1 Volatile 字段类型

`volatile` 字段的类型会被转换为 `std.sync` 中相应的原子类型。对这些字段的读取和写入操作会被转换为这些类型的 `load` 和 `store` 成员函数的调用。

[引用类型](#)的映射取决于翻译模式, 因为在语义保留模式 (`-mode=semantic`) 下, 它们会被 `Option<T>` 包装。**注意:** 由于 `std.sync` 中尚未提供字符和浮点数值类型的原子类, 因此它们必须被包装到容器中, 从而实际上成为引用类型。

在下表中，***X'*** 是 J2CJ 将 Java 类型 *X* 翻译为仓颉类型时所使用的类型，其中 Java 类型 *X* 用于除 **volatile** 字段类型以外的其他上下文。

在默认的“代码风格优先”模式（`-mode=codestyle`）下，**volatile** Java 字段的类型会按以下方式转换：

Java	Cangjie
boolean / Boolean	AtomicBool
byte / Byte	AtomicUInt8
short / Short	AtomicInt16
int / Integer	AtomicInt32
long / Long	AtomicInt64
char / Character	AtomicReference<Box<Rune>>
float / Float	AtomicReference<Box<Float32>>
double / Double	AtomicReference<Box<Float64>>
String	AtomicReference<Box<String>>
class C	AtomicReference<C'>
interface I	AtomicReference<I'>
T[]	AtomicReference<Box<Array<T'>>>

在语义保留模式（`-mode=semantic`）下，**volatile** Java 字段的类型将按以下方式转换：

Java	Cangjie
boolean	AtomicBool
byte	AtomicUInt8
short	AtomicInt16
int	AtomicInt32
long	AtomicInt64
char	AtomicReference<ValueBox<Rune>>
float	AtomicReference<ValueBox<Float32>>
double	AtomicReference<ValueBox<Float64>>
Boolean	AtomicOptionReference<ValueBox<Bool>>
Byte	AtomicOptionReference<ValueBox<UInt8>>
Short	AtomicOptionReference<ValueBox<Int16>>
Integer	AtomicOptionReference<ValueBox<Int32>>
Long	AtomicOptionReference<ValueBox<Int64>>
Character	AtomicOptionReference<ValueBox<Rune>>

Java	Cangjie
Float	AtomicOptionReference<ValueBox<Float32>>
Double	AtomicOptionReference<ValueBox<Float64>>
String	AtomicOptionReference<ValueBox<String>>
class C	AtomicOptionReference<C'>
interface I	AtomicOptionReference<I'>
T[]	AtomicOptionReference<ValueBox<Array<T'>>>
NOTE: If T is a Java reference type other than array, T' is the respective Cangjie type wrapped in Option.	

4.4.2 函数类型

在默认的“代码风格优先”模式（`-mode=codestyle`）下，带有`@FunctionalInterface`注解的 Java 函数式接口类型的使用会被转换为等效的 Cangjie 函数类型的使用，除非以下任一或两者同时成立：

- 接口的唯一抽象方法是参数化的。

注意：如果方法的声明包含一个或多个类型 `_ 参数 _`，即使其签名中不包含类型 `_ 变量 _`，该方法也被视为参数化的：

```
@FunctionalInterface
interface FI {
    <T> int foo(int arg); // 尽管没有类型变量，但仍然是参数化的
}

@FunctionalInterface
interface PFI <T> {
    int bar(int arg);      // 不是参数化的（即使接口是参数化的）
}
```

- 由于接口类型声明是递归的，因此无法定义函数类型：

```
@FunctionalInterface
interface Recursive {
    Recursive f();
}
```

例如，`java.lang.Runnable`的使用被转换为`() -> Unit`，`java.util.function.Function<String, String>`的使用被转换为`(String) -> String`，依此类推。因此，以这种方式转换的函数式接口的抽象方法的任何调用都会被转换为对函数的调用。函数式接口的声明仍然会被转换，无论模式如何，但`@FunctionalInterface`注解不会被保留。实现函数式接口的类（包括匿名类）的定义会正常转换，但在现在期望相应函数类型值的上下文中，这些类的 `_ 实例 _` 的任何使用都会被转换为成员函数引用。

```
@FunctionalInterface
interface DoubleFunc {
    double f(double x);
}

class Halver implements DoubleFunc {
    public double f(double x) { return x / 2.0; }
}

. . .
static double apply(DoubleFunc df, double x) {
```

```

        return df.f(x);
    }
    . . .
    double res1 = apply(x -> x * 2.0, 10.0);
    double res2 = apply(new Halver(), 10.0);

```

```

interface DoubleFunc {           // 接口声明被保留
    func f(x: Double): Double
}

class Halver <: DoubleFunc {
    override public func f(x: Double): Double {
        return x / 2.0;
    }
}

// 参数'df'具有函数类型，而不是接口类型...
static func apply(df: (Double) -> Double, x: Double): Double {
    df(x)                          // ...并且直接调用
}

let res1 = apply({x => x * 2.0}, 10.0)
let res2 = apply(Halver().f, 10.0)

```

4.5 转换结果

J2CJ 最后输出的转换统计信息包括：转换器成功生成的代码行数，输出的总行数（即成功生成的代码行数 加上 带有**标记**的行数），以及这两个数字的比率。

```
Translation ratio: 128/130 lines (98,5%)
```

如果转换不成功（即比率小于 100%），则需要检查生成的代码并处理每个标记。更多详细信息，请参阅[高级用法](#)。

如果转换成功（即转换率为 100%），则可以使用标准 **cjpm** 工具编译生成的仓颉源代码，如下所示：

1. 转换器会将 **cjpm.toml** 文件放置于生成的模块根目录中：

```

[dependencies]
[dependencies.j2cjlib]
  path = "../j2cjlib"
  version = "1.0.0"

[package]
  . . .

```

注意：对 `../j2cjlib` 模块的依赖仅在 J2CJ 以【语义保留】模式（即 `--mode=semantic`）运行时才会出现。该模块本质上是一个辅助库，随翻译器一起提供。它提供了一些在标准仓颉库中没有对应项的类型和函数。你需要将其从 J2CJ 的安装目录中复制到 J2CJ 的输出目录中，或者创建一个符号链接：`ln -s /path/to/j2cj/j2cjlib/ ../j2cj/output/dir`

2. 这样就可以直接在包含转换后源代码的 **cjpm.toml** 文件所在目录运行 `cjpm build`。

5 映射

Java 语言（如声明、语句、运算符、原生类型等）的转换是一项复杂的工作，仓颉标准库已提供了大量与核心 Java API 类似的类和接口（如 `String`、`HashMap` 或 `Iterable`），可以直接映射到仓颉类型。

J2CJ 通过建立 Java 类型与仓颉等效项之间的映射，确保类型或成员的使用能够得到准确转换。J2CJ 内置了大量核心 Java API 类的映射，并支持用户自定义映射，包括个人开发的类和第三方库，从而扩大自动转换代码的范围。

用户可创建的映射被视作基本的名称匹配，只需确保仓颉函数的参数数量（必要）和顺序与原始 Java 方法保持一致。

注意：方法的具体语义可能会有所不同。例如，在 Java 中获取给定 `java.lang.String` 实例长度的无参数方法是 `length()`，而在仓颉中获取 `String` 长度的对应方法是 `size()`。Java 返回的是 UTF-16 格式的字符数，而仓颉返回的是 UTF-8 格式的字符数。尽管将 `length` 映射到 `size` 保留了某些不变量，如 `(s1 + s2).length() == s1.length() + s2.length()`，但转换后可能导致依赖 `String.length` 方法精确语义的 Java 代码失效。

对于部分未直接支持的核心 Java API 类型和方法，J2CJ 工具也提供了内置的、更复杂的映射。

5.1 映射加载过程

启动时，J2CJ 首先在当前工作目录下寻找名为 `root.cjmap` 的文件。如果未能找到，J2CJ 将从其包含的 `j2cj.jar` 包中加载并解析内置的 `root.cjmap` 文件。

`root.cjmap` 包含以下两个部分：

- 内置默认仓颉类型和成员的定义（可参考 `j2cj.jar` 中的 `stdlib.cjmap` 文件）
- Java 至仓颉映射的定义（可参考 `j2cj.jar` 中的 `mappings.cjmap` 文件）

默认的 `root.cjmap` 文件仅包含以下内容：

```
#include "stdlib.cjmap"
#include "mappings.cjmap"
```

注意：IntelliJ IDEA 插件会自动查找所有项目或模块根目录下的 `.cjmap` 文件并将其文件路径传给 J2CJ。

5.2 添加自定义映射

开发者可以使用以下任一方式添加自定义映射。

方式 1：

开发者可以通过命令行的方式直接将 `.cjmap` 文件的路径传给 J2CJ：

```
/path/to/JDK/bin/java -Dmodule.name=App -jar /path/to/j2cj/j2cj.jar \
  App.java custom.cjmap
```

J2CJ 启动时会首先加载 `root.cjmap` 文件，然后会按照顺序，依次加载命令行中出现的 `.cjmap` 文件。

J2CJ 提供的 IDE 插件会自动搜索位于项目或模块根目录中的 `.cjmap` 文件的路径名，并自动传递给 J2CJ 工具作为自定义映射文件的输入。

方式 2：

- 在 J2CJ 启动目录中创建一个本地的 `root.cjmap` 文件，其内容与默认文件相同。


```
#include "stdlib.cjmap"
#include "mappings.cjmap"
```

- 在同一目录下创建一个新的 `.cjmap` 文件，并在其中添加自己的映射。假设将其命名为 `custom.cjmap`。
- 添加以下内容：

```
#include "custom.cjmap"
```

至本地的 `root.cjmap` 文件中。

后续升级 J2CJ 版本时，用户只需检查默认的 `root.cjmap` 文件是否有更改。如有，将其同步至本地 `root.cjmap` 文件中即可。

5.3 映射语法

仓颉的限定名称格式如下：`moduleName.packageNameSeparatedByDots.packageMember`。其中，`packageMember` 为类型名称。

5.3.1 仓颉声明

在 J2CJ 映射文件中，仓颉定义的语法接近仓颉语法的子集，但要求每个类型名称必须完全限定，[如上所述](#)。这种限定有助于省略不必要的信息，包括无需转换的元素，即：

- 无导入语句
- 无方法体
- 无字段初始化器
- 无 `var` 或 `let` 关键字

示例：

Option 枚举：

```
enum std.core.Option<T> {
    getOrThrow(): T
    isSome(): Bool
    isNone(): Bool
    static func Some(value: T): Option<T>
    static func None(): Option<T>
}
```

Iterable 和 Iterator 接口：

```
interface std.core.Iterable<E> {
    iterator(): std.core.Iterator<E>
}

interface std.core.Iterator<E> <: std.core.Iterable<E> {
    next(): Option<E>
}
```

Collection 接口：

```
interface std.core.Collection<T> <: std.core.Iterable<T> {
    size: Int64
}
```

```

    isEmpty(): Bool
    toArray(): Array<T>
}

```

带ToString扩展的Array结构:

```

struct std.core.Array<T> <: std.core.Collection<T> {
    copyTo(Array<T>, Int64, Int64, Int64): Unit
    clone(): Array<T>
}
extend std.core.Array<T> <: std.core.ToString where T <: std.core.ToString { }

```

ToString:

```

interface std.core.ToString {
    toString(): String
}

```

类:

```

class std.sync.Monitor {
    wait(): Bool
    notify()
    notifyAll()
}

```

包级函数声明:

```

package std.math {
    func cbrt(x: Float64): Float64
    func cbrt(x: Float32): Float32
    func cbrt(x: Float16): Float16
    func logBase(x: Float32, base: Float32): Float32
    . . .
}

```

5.3.1.1 扩展声明

扩展只能在相应的**package**声明的主体中声明。参考 J2CJ 附带的stdlib.cjmap实例进行说明:

```

. . .
public class std.io.StringReader<T> where T <: std.io.InputStream {
    public init(input: T)
    . . .
}

. . .
package std.io {
    extend<T> std.io.StringReader<T> <: Resource where T <: Resource {
        public func close(): Unit
        public func isClosed(): Bool
    }
    extend<T> std.io.StringReader<T> <: std.io.Seekable where T <: std.io.Seekable {
        public func seek(sp: std.io.SeekPosition): Int64
    }
}

. . .
package j2cjlib.utils {
    . . .
    extend<T> std.io.StringReader<T> <: j2cjlib.utils.J2CjStringReaderExtension {
        public func read(buf: Array<Rune>): Int32
    }
}

```

```

    }
    . . .
}

```

5.3.2 Java 映射

映射文件的第二部分定义了 Java 与仓颉之间的映射关系。任何被转换源代码文件中未定义的 Java 类或接口，若无以下对应映射，则无法转换：

- 类型本身（类型参数的数量、顺序及名称一致），除非只需要将其 **static** 成员映射为包级函数和/或全局变量。
- 成员或构造函数的定义包含在被转化的源代码中。

注意：某些核心 Java API 方法无法被映射/转换。例如 `java.util.Iterator<E>.hasNext()` 方法。仓颉接口的 `core.Iterator<E>` 使用 `Option` 来通知调用者元素为空，与 Java 的实现机制不同，无法做相同语义的转化。

5.3.2.1 类型映射

单个 Java 类型映射的语法如下：

```

'mapping' Java-type [ type-arguments ] [ '=>' Cangjie-type [ type-arguments ] ] '{'
  { Java-member [ '=>' Cangjie-member ] }
'}'

```

其中 `Java-type` 和 `Cangjie-type` 分别是 Java 和仓颉类型的完全限定名称。对于 `std.core` 包中的仓颉类型全限定名称会被忽略。

`'=>' Cangjie-type [ type-arguments ]` 是可选的。它支持映射那些本质上是命名空间的 Java 类，即这些类只有 **static** 成员，没有可访问的构造函数，因此在转换后不会实例化。而仓颉通过全局变量和高级函数提供等效功能，因此并不存在可映射的仓颉类。例如，`java.lang.Math` 类。

```

mapping java.lang.Math {
    . . .
}

```

注意：可以将 Java 接口映射到仓颉类，但应作为仓颉中缺少相应接口时的第二选择。

从技术上讲，将 Java 类映射到仓颉接口也可以实现，但这种映射用处不大，因为实例化的类无法转换。

5.3.2.2 成员和构造函数映射

在最简单的场景，目标 `Cangjie-type` 只有一个与之同名且语义相同的成员，因此对应的成员映射仅需包含名称：

```

mapping java.lang.Iterable<T> => std.core.Iterable<T> {
    iterator
}

```

同样的，有对等构造函数的映射场景：

```

mapping java.util.HashSet<E> => std.collection.HashSet<E> {
    <init>
}

```

如果相应的 `Cangjie-type` 成员具有不同的名称，可以在 `=>` 后指定该名称。

```
mapping java.util.Set<E> => std.collection.Set<E> {
    add => put
    addAll => putAll
    . . .
}
```

其他情况，`Cangjie-member`可作为一个完全限定名，映射到另一类型的成员，甚至是映射到仓颉包的全局变量或高级函数。这对于映射 Java 的 **static** 字段和方法尤其有用。

```
mapping java.lang.Math {
    abs => std.math.abs
    sqrt => std.math.sqrt
    exp => std.math.exp
    . . .
}
```

重要提示：目标方法或构造函数可能具有不同但兼容的参数类型或返回类型。J2CJ 会自动执行必要的类型转换。

5.4 其他方法和构造函数映射注意事项

5.4.1 映射重载方法和构造函数

上述映射形式同样适用于重载的 Java 方法和构造函数。请谨慎使用，可能会引入潜在问题。

并非所有重载的 Java 方法/构造函数在目标仓颉类中都有对应的映射。例如 `HashSet` 类映射：

```
mapping java.util.HashSet<E> => std.collection.HashSet<E> {
    <init>
}
```

在 J2CJ 转化 Java 代码的过程中，任何使用构造函数 `HashSet(int initialCapacity, float loadFactor)` 的地方都会被**标记**，因为在仓颉的 `HashSet` 类中没有对应的构造函数。如果这个构造函数在代码中并未实际使用，那么不会引起问题，而且也不会产生警告。

因此，最安全的做法是逐个指定重载方法和构造函数的映射，并使用签名来区分它们，直到仓颉中的 Java 方法都具有唯一名称，即没有发生方法重载。

Example:

仓颉线程 API 与 Java 的线程 API 有根本性的不同，因此 J2CJ 辅助库应运而生。不过，它并不支持线程组，因此在 Java 21 中，`java.lang.Thread` 的九个构造函数中只有四个可以被映射：

```
mapping java.lang.Thread => j2cjlib.mappings.sync.J2CjThread {
    <init>()
    <init>(java.lang.Runnable)
    <init>(java.lang.String)
    <init>(java.lang.Runnable, java.lang.String)
    . . .
}
```

5.4.1.1 Java 签名

映射文件中 Java 方法和构造函数的签名类似于 Java 方法头而非 JVM 签名。这些签名使用点 (.) 而非斜杠 (/) 作为包名分隔符，使用完整的初始类型名称，使用 [] 表示数组，以及使用逗号分隔参数。

- 比如签名为 `(java.lang.Object, java.lang.Integer) int` 而非 `(Ljava/lang/Object;Ljava/lang/Integer;)I`，为 `(byte[]) long` 而非 `([B)J`。

方法签名需要以返回类型或**void**结尾。

所有签名都遵循 Java 语言规范进行擦除：把类型变量都替换为上界，并移除所有类型参数列表。例如：

- `java.util.HashSet`构造函数的参数类型从`java.util.Collection<? extends E>`简化为`java.util.Collection`
- `java.util.Arrays.parallelSort`的参数类型`T[]`，其中带有泛型约束的`T extends java.lang.Comparable<? super T>`简化为`java.lang.Comparable[]`。

5.4.2 映射继承的方法

重要提示：无需为继承的方法定义映射。

映射给定的方法时，只需要为那些引入或重写该方法的类和接口提供映射。用户也可以为仅继承该方法的类型进行映射，但在当前版本中，这些映射将被忽略。

这意味着即使程序只使用了实现类，也可能需要为抽象类的非抽象方法定义映射。

示例：

```
abstract class AbstractFoo {
    abstract int foo();
    boolean bar() { return false; }
    char baz() { return 'A'; }
}

class ConcreteFoo extends AbstractFoo {
    int foo() { return 1; }
    @Override
    boolean bar() { return true; }
}
```

即使被转换的程序只使用了`ConcreteFoo`类，下面的映射仍然是不完整且冗余的：

```
mapping ConcreteFoo => CJ_ConcreteFoo_Equivalent {
    foo
    bar
    baz
}
```

原因是，转换器在查找`new ConcreteFoo().baz()`类似代码片段中的`.baz()`适用映射时，就会发现调用的是`AbstractFoo.baz()`。于是它会在`AbstractFoo`的成员映射中查找`baz`的映射。

正确映射如下所示：

```
mapping AbstractFoo => CJ_AbstractFoo_Equivalent {
    baz
}

mapping ConcreteFoo => CJ_ConcreteFoo_Equivalent {
    foo
    bar
}
```

5.5 更多简单映射示例

有关更多示例和灵感，请查看`j2cj.jar`中包含的`mappings.cjmap`文件。

注意：该文件尚未完善。目前，J2CJ 的焦点在于 Java 平台的 API 映射，并借助复杂的内部映射系统转换仓颉标准库中暂未直接对应的 Java API。因此，该文件并没有包含 J2CJ 提供的所有支持的映射。

6 注解支持

J2CJ 支持自定义注解类型的转换，也支持部分 Java 标准注解的转换。

6.1 Java 标准注解类型

- 注解@SafeVarargs、@Documented、@Inherited和@Repeatable在当前的仓颉中并没有等效项，因此会被转换为**标记**。未来若提供了等效项，用户便可以添加简单的映射，映射方法详见下方的**注解映射**。
- 注解@Override会被“强制转换”为override修饰符。
- 注解@SuppressWarnings会被识别并跳过。
- 注解@Retention会被跳过。所有自定义注解默认采用运行时保留策略。
- 注解@Target被转换成仓颉@Annotation注解的target参数,并支持以下元素类型:TYPE、ANNOTATION_TYPE、FIELD、METHOD、PARAMETER和CONSTRUCTOR。其他元素类型PACKAGE、LOCAL_VARIABLE、TYPE_PARAMETER、TYPE_USE、MODULE和RECORD_COMPONENT由于没有仓颉等效项，会被转换为标记。
- 在默认的“代码风格优先”模式(-mode=codestyle)下,当函数式接口被翻译时,会考虑@FunctionalInterface注解，但该注解不会以任何方式传播到输出的仓颉代码中。在**语义保留**模式(-mode=semantic)下，该注解会被识别并跳过。详情请参阅**函数式接口**。
- @Deprecated注解会被翻译为其仓颉等效注解，名称保持不变。

6.2 自定义注解类型

6.2.1 注解类型声明

Java 注解类型声明可以被转换为仓颉注解类。在 Java 中通过@interface成员方法声明定义的元素会被处理成两个与元素同名的实体，即一个成员变量和一个 const init 参数（可能有默认值）。

示例：

```
@Retention(value= RetentionPolicy.RUNTIME)
@Target({ ElementType.TYPE, ElementType.FIELD, ElementType.METHOD, ElementType.
    CONSTRUCTOR, ElementType.PARAMETER })
public @interface Anno {
    boolean bool();
    String name() default "name";
    String color() default "color";
}
```

转换为：

```
@Annotation[target: [Type, MemberVariable, MemberFunction, Init, Parameter]]
public class Anno {
    let bool: Bool

    let name: ?String

    let color: ?String

    const init(bool: Bool, name!: ?String = "name", color!: ?String = "color") {
        this.bool = bool
        this.name = name
    }
}
```

```

        this.color = color
    }
}

```

6.2.2 使用注解

将 Java 注解转换为调用相应仓颉注解类的 `const init` 函数：

```

@Anno(bool = true)
public class Annotated {
    @Anno(bool = true, name = "f")
    int field;
}

```

转换为：

```

@Anno[true]
public open class TestAnno <: Hashable & ToString {
    @Anno[true, name: "f"]
    var field: Int32 = 0
}

```

6.3 注解映射

目前，J2CJ 仅包含对显式支持的 Java 注解类型的简单映射，如有需要，用户可以自行创建映射。

要在 `.cjmap` 文件中声明一个仓颉注解类，请使用仓颉语法列出所有成员变量和构造函数的签名。

```

@Annotation class std.core.NonExistentCangjieLabel {
    init(cjvalue!: String = "cjvalue")
    cjvalue: String
}

```

要定义 Java 库注解的简单映射（“简单”是指每个 Java 注解元素都有合适的映射），请使用常规的 `mapping` 声明：

```

mapping jdk.jfr.Label => std.core.NonExistentCangjieLabel {
    value => cjvalue
}

```


7 J2CJ 不支持的 Java 特性

技术上来说，对于任何正确的 Java 程序，都会存在一个与之相对的正确的小数程序，如果两者输入相同，输出也会相同。然而，由于 Java 与小数的语法、语义、标准库实现并非完全对应，因此转换工具需使用额外处理来支持转化。

7.1 词法结构

7.1.1 注释

J2CJ 会保留原始代码中的注释，但：

- 注释对齐方式不被保留。

```
private int x = 153      // 153
    * 2                // 乘以2
    /// 7              // 不被7整除
    -/* 6 */5          // 减去6^H^H^H5
    /* + randomInt(10) // 混入一个随机整数 */
    + 1;              // 加1
```

转换为：

```
private var x = 153i32 // 153
* 2i32 // 乘以2
    /// 7          // 不被7整除
- /* 6 */ 5i32 // 减去6^H^H^H5
/* + randomInt(10) // 混入一个随机整数 */ + 1i32 // 加1
```

- 不保证转化后注释的位置与转化前一致。例如，Java 的 **if** 语句和循环语句并不一定有 { } 块。

```
while(f(x) > epsilon) x /= 2.0;
if (p(y)) do_this(y) else do_that(y);
```

在小数中，条件表达式和循环表达式始终包含 { } 块。

参考以下两个 Java 语句：

```
if (foo == null) foo = FooFactory.make(); // 在需要时创建 foo

if (bar != null) myBar = bar
else throw new NullPointerException(); // Should never happen
```

每行注释都放在 **if** 语句末尾分号 (;) 后面。第一个注释与整个语句有关，第二个注释与 **else** 部分有关。转换器无法区分这两种情况，因此它总是假设紧跟在语句后面的行注释与整个语句相关。

```
if (bar.isSome()) {
    myBar = bar
} else {
    throw NoneValueException()
} // Should never happen
```

更改原始 Java 代码可以解决这个问题。

```
if (bar != null) myBar = bar
else {
    throw new NullPointerException(); // should not reach here
}
```

会转换为：

```
if (bar.isSome()) {
    myBar = bar
} else {
    throw NoneValueException() // should not reach here
}
```

- 任何位于文件末尾声明之后的注释都会丢失：

```
class GetsTranslated {}
// class LostInTranslation {}
<EOF>
```

- 对于文件头部注释，J2CJ 遵循以下启发式规则：如果注释为 Javadoc 注释，则认为其与后续类或接口声明相关；否则，将其视作文件头注释。以下是两种具体示例：

```
/*
 * 包含版权和许可的头部注释
 */

class Foo {
    . . .
}
```

转换为：

```
/*
 * 包含版权和许可的头部注释
 */

package UNNAMED

class Foo {
    . . .
}
```

而

```
/**
 * 代表foo的类。
 * <p>
 * 实现通用的foo，必须包含bar，并且可能链接到baz。
 */

class Foo {
    . . .
}
```

转换为：

```
package UNNAMED

/**
 * 代表foo的类。
 * <p>
 * 实现通用的foo，必须包含bar，并且可能链接到baz。
 */
class Foo {
    . . .
}
```

7.1.2 字面量

7.1.2.1 数值字面量

数值字面量的基数信息在翻译过程中丢失了。所有从 **Java** 翻译过来的数值字面量都是十进制（基数为 10）。

小数部分中带有下划线和尾随零的格式也会丢失。

7.1.2.2 字符串字面量

字符串字面量中的 Unicode 替代字符（`\ud800-\udfff`）目前会被原样转换。

7.2 类型、值和变量

7.2.1 引用类型

在仓颉中不存在空引用这一概念，因此在默认的“代码风格优先”模式（`-mode=codestyle`）下，J2CJ 假定所有 **Java** 引用类型的实体都是非空的，并为任何显式使用 **null** 的情况生成 **标记**，以便开发者在翻译后适当地审查和编辑生成的仓颉代码：

```
class Foo {
    void baz(Foo bar) {
        Foo foo = null;
        if (bar != null) baz(null);
        Foo qux = new Foo();
    }
}
```

```
class Foo {
    init() { }
    func baz(bar: Foo): Unit {
        let foo = <-- null -->
        if (bar != <-- null -->) {
            baz(<-- null -->)
        }
        let qux = Foo()
    }
}
```

在 **语义保留** 模式（`-mode=semantic`）下，所有翻译后的引用类型都会被包装在 `Option<T>` 中，其中 **None** 表示空引用，除非翻译器能够证明某个值不为 **null**：

```
// -mode=semantic (为清晰起见省略了样板代码)
open class Foo <: Equatable<Foo> & Hashable & ToString {
    init() { }
    public open func baz(bar: ?Foo): Unit {
        let foo: ?Foo = None
        if (bar.isSome()) {
            baz(None)
        }
        // Java 中的 `new T()` 操作符要么创建 T 的实例，要么抛出异常，
        // 因此它永远不会评估为 null:
        let qux = Foo()
    }
    . . .
}
```

7.2.2 参数化类型

当前的 J2CJ 中，类型参数边界在大部分情况下会被转换为不支持标记。未来版本将会对此进行优化。

7.2.3 交集类型

在类型边界得到支持之前，交集类型也无法支持。

7.3 转换和上下文

7.3.1 装箱转换

在仓颉中，不需要对值类型进行装箱，因为它们可以直接用于类型参数化，这与 Java 不同。因此，在默认的“代码风格优先”模式（`-mode=codestyle`）下，原始类型的 Java 包装器（如 `java.lang.Integer`）通常会被转换为相应的仓颉值类型，除非适用的映射要求使用可选值：

```
Integer baz = Integer.valueOf("42");
Integer qux = Integer.valueOf(42);
```

会被转换为：

```
let baz = Int32.parse("42")
let qux = 42i32
```

在语义保留模式（`-mode=semantic`）下，会像引用类型一样使用 `Option<T>` 包装。

```
public open func bar(): Unit {
    let baz: ?Int32 = Int32.parse("42")
    // 当前版本无法理解
    // Integer.valueOf(int i) 不能返回 null:
    let qux: ?Int32 = 42i32
}
```

7.3.2 拆箱转换

由于装箱转换并未真正被翻译，拆箱转换也是如此。如果在语义保留模式下使用了 `Option<T>` 包装（`-mode=semantic`），则通过 `getOrThrow()` 实例成员函数来检索值。

7.4 名称

尽管不推荐，但 Java 允许字段、成员类型和方法使用相同的名称（标识符）。

但在仓颉中，类型的成员变量和函数必须有不同的名称。（仓颉没有成员类型，因此也没有成员类型的名称冲突）此外，实例成员函数和静态成员函数必须有不同的名称。

因此，如果名称与其他字段/方法名称冲突，转换器会将“Method”追加到实例方法名称后，并将“Static”追加到静态方法名称后。

```
int qux;
void qux() {}
static void qux(int x) {}
```

转换为：

```
var qux: Int32 = 0
public open func quxMethod(): Unit { }
static func quxStatic(x: Int32): Unit { }
```

7.4.1 完全限定名称

在 Java 中，可以通过完全限定名直接访问非本包的公共成员，无论此成员是否导入。而在仓颉中，此类操作是禁止的。J2CJ 会在这种情况下使用简单名称，但这可能导致不可预见的名称冲突问题。

7.4.2 范围

7.5 包

7.5.1 静态导入

由于仓颉不识别 **static** 成员的导入，Java 中的 **import static** 声明会被转换为普通的仓颉导入声明。

7.6 类

7.6.1 类声明

7.6.1.1 类修饰符

在仓颉中，对于类的 **strictfp** 修饰符会被跳过。所有的浮点计算默认使用该模式。

sealed 类会被转换为普通类。

7.6.1.2 内部类和封闭实例

J2CJ 通过放宽对内部类字段和方法访问修饰符的限制以支持内部类。这些字段和方法通过 **javac** 生成的访问方法进行引用。

7.6.2 字段声明

7.6.2.1 transient 字段

不支持。仓颉中没有与 **transient** 修饰符对应的等效项。这类字段将被忽略，并生成一个标记。这可能会影响依赖于该字段在反序列化后具有默认值的代码实现。

7.6.2.2 volatile 字段

现在支持使用 **std.sync.Atomic*** 类来处理 **volatile** 字段，但它们在默认的“代码风格优先”模式和“语义保留优先”模式下的转换方式有所不同。有关详细信息，请参阅 **Types of volatile Fields**。目前尚不支持对基本类型 **volatile** 字段执行以下操作：

- 单目运算符：

```
volatile int a;
    .   .   .
    a++;
```

- 复合赋值运算符:

```
volatile int a;
    .   .   .
    a += 10;
```

这些操作会在输出代码中插入相应的**标记**。

7.6.3 方法声明

7.6.3.1 方法修饰符

对于方法的**strictfp**修饰符会被跳过。所有的浮点计算默认使用该模式。

7.6.4 实例初始化器

由于仓颉的限制，我们不支持也不计划支持实例初始化器。目前的 J2CJ 版本生成特定的标记。

7.6.5 静态初始化器

我们不支持也不计划支持静态初始化器，主要是因为 Java 和仓颉在语义上有较大差异。目前的 J2CJ 版本生成特定的标记。

7.6.6 枚举类型

Java 枚举类型与仓颉枚举的语义差异较大，以至于将前者翻译为后者往往不切实际。Java 枚举类型本质上是一个特殊的类，因此它可能具有带参数的非默认构造函数、定义为匿名类实例的常量等。如此复杂的 Java 枚举无法合理地翻译为仓颉枚举，后者是一种求和类型。然而，在最简单的情况下，翻译为仓颉枚举是可行的。在默认的“代码风格优先”模式（**-mode=codestyle**）下，J2CJ 将仅包含命名常量列表的 Java 枚举类型翻译为仓颉枚举。另请参阅**--enum-functions**命令行选项。否则，包括在**语义保留**模式（**-mode=semantic**）下翻译此类简单的 Java 枚举类型，Java 枚举类型将被翻译为具有私有**init**函数和表示枚举常量的**static let**字段的常规仓颉类。它们通常可用，但您可能需要考虑手动将翻译后的代码迁移到适当的仓颉枚举中。

7.6.7 记录类

不支持记录类。

7.7 接口

7.7.1 接口声明

对于接口的**strictfp**修饰符会被跳过。所有的浮点计算默认使用该模式。

7.7.2 字段（常量）声明

Java 接口可以包含字段，这些字段隐式地是 **public**、**static** 和 **final** 的。这样的字段必须伴随初始化表达式，尽管这些表达式不一定是常量。相比之下，Cangjie 接口根本不能包含字段。作为一种变通方法，J2CJ 将接口常量转换为只读的静态属性：

```
interface Computer {
    int THE_ANSWER = 42;
}
```

```
interface Computer {
    static prop THE_ANSWER: Int32 {
        get() {
            42
        }
    }
}
```

如上例所示，简单的字面量初始化表达式支持所有类型的接口。不支持有界类型参数接口的复杂初始化表达式；此时会生成一个标记：

```
interface I1 {
    public static String FIZZ = "Fizz"; // OK
}

interface I2<T> extends I1 {
    String FIZZBUZZ = FIZZ + "Buzz"; // OK
}

interface I3<T> extends Comparable {
    String FIZZBUZZ = I1.FIZZ + "Buzz"; // Marker will be generated
}
```

```
interface I1 {
    static prop FIZZ: String {
        get() { "Fizz" }
    }
}

private let I2_FIZZBUZZ = I2<Any>.FIZZBUZZ_init
interface I2<T> <: I1 {
    static prop FIZZBUZZ: String {
        get() { I2_FIZZBUZZ }
    }

    static prop FIZZBUZZ_init: String {
        get() { FIZZ + "Buzz" }
    }
}

private let I3_FIZZBUZZ = <-- Constants in interfaces with bounded type parameters
are unsupported: I3 -->.FIZZBUZZ_init
interface I3<T> where T <: Comparable<Any> {
    static prop FIZZBUZZ: String {
        get() { I3_FIZZBUZZ }
    }

    static prop FIZZBUZZ_init: String {
```

```

        get() { "Fizz" + "Buzz" }
    }
}

```

7.7.3 方法声明

支持接口方法 **static** 修饰符和 **default** 修饰符。然而，调用子类中重写方法的默认实现在仓颉中无法直接表达，因此不支持。

7.7.4 注解类型

7.7.4.1 预定义注解类型

支持部分预定义注解类型。详情见预定义注解类型。

7.7.5 函数式接口

支持功能接口，但根据其属性和翻译模式的不同进行不同的翻译。在默认的“代码风格优先”模式（`-mode=codestyle`）下，某些 Java 功能接口类型的使用会被翻译为等效的 Cangjie 函数类型的使用。这进而使得 Java 的 **lambda 表达式** 能够被翻译为 Cangjie **lambda**。在所有其他情况下，函数接口的翻译方式与所有其他接口相同。`@FunctionalInterface` 注解永远不会被保留。详情请参阅 [函数类型](#)。

7.8 块和语句

7.8.1 标签语句

不支持 Java 标签语句。

7.8.2 switch 语句

支持单个 `SwitchBlockStatementGroup` 中存在多个 `SwitchLabels`，但不支持穿透（fall through）。

注意：当前版本的 J2CJ 会将每个不以 **break**、**return** 或 **throw** 语句结尾的 `_SwitchBlockStatementGroup_` 语句组，当作以 **break** 语句结尾来处理，并且不会产生告警。因此，生成的仓颉代码是不正确的（但仍然可能编译通过）。这是一个已知问题，将会被修复。

7.8.3 基础 for 语句

Java 中的基础 **for** 语句在仓颉中没有直接对应的等效项，因此，在一般情况下，被翻译为仓颉的 **while** 循环表达式：

```

double sum = 0.0;
for(double x = 1.0; x > 0.0; x = x/2.0) {
    sum += x;
}

```



```
var sum = 0.0
var x = 1.0
while (x > 0.0) {
    sum += x
    x = x / 2.0
}
```

由于需要将原始 **for** 语句中 `_ForInit_` 部分的变量声明翻译成代码，并且这些翻译后的变量声明必须插入到 **while** 循环表达式之前，因此这些变量的名称可能会与在同一 Java 方法中稍后声明的局部变量的名称发生冲突。在这种情况下，J2CJ 会额外地将整个 **while** 循环表达式包装在一个 **do-while** 循环表达式中，该 **do-while** 循环仅执行一次，从而创建一个嵌套的作用域。

```
double sum = 0.0;
for(double x = 1.0; x > 0; x = x/2) {
    sum += x;
}

// The scope of this 'x' is from the point of declaration forward
// in both Java and Cangjie:
int x = 42;
```

```
var sum = 0.0
do {
    var x = 1.0
    while (x > 0.0) {
        sum += x
        x = x / 2.0
    }
} while (false)
let x = 42i32
return sum

// ← This do-while introduces a nested scope...
// ← ...for this 'x' to not clash with the 'x' below

// ← Here the nested scope ends
```

J2CJ 现在可以将某些“规范”形式的基本 **for** 语句翻译为仓颉 **for-in** 循环表达式，用于遍历一个范围：

```
int sum = 0;
for (int x = 1; x <= 10; x++) {
    sum += x;
}
```

```
var sum = 0i32
for (x in 1i32..=10i32) {
    sum += x
}
```

考虑 Java 基本 **for** 语句的语法：

for ([ForInit] ; [Expression] ; [ForUpdate]) Statement

为了让 J2CJ 将给定的基本 **for** 语句转换为对某个范围的 **for-in** 循环表达式，该语句必须满足以下所有条件：

- *ForInit* 部分必须声明一个且仅有一个基本类型的有符号整数变量：**byte**、**short**、**int** 或 **long**。

假设该变量的名称是 *x*。

- 表达式 必须是 *x* 与某个表达式 *E* 的计算结果之间的数值比较。换句话说，它必须具有以下两种形式之一：*xOp(E)* 或 *(E)Opx*，其中 *Op* 是数值比较运算符 *<*、*<=*、*>* 和 *>=* 之一。如果括号不会改变计算顺序，则可以不使用括号来包围 *E*。
- 在仅使用语义保留模式 (`-mode=semantic`) 的情况下，表达式 *E* 还必须是不可变的。当前版本的 J2CJ 仅在此上下文中将以下表达式视为不可变：字面量、方法参数、**final** 变量以及实际上为 **final** 的变量。

注意 在这种模式下，J2CJ 还应该检查在 `_ForInit` 部分声明的变量是否在 `Statement_` 中的任何地方被修改。然而，目前这一检查尚未实现，因此当前版本可能会生成无法编译的仓颉源代码。一种可能的解决方法是手动将有问题的 `for-in` 循环表达式替换为 `while` 循环。这个问题将在未来的版本中得到解决。

- `ForUpdate` 部分必须包含且仅包含一条语句，该语句要么对变量 `x` 进行减法、加法操作，要么将一个数值字面量的值加到或减去变量 `x`。换句话说，该语句必须具有以下形式之一：

```

++x
x++
x--
--x
x += Literal
x -= Literal
x = x + Literal
x = Literal + x
x = x - Literal

```

其中，`Literal` 是一个数字字面量。

7.8.4 增强 `for` 语句

仓颉完全支持 Java 的增强 `for` 语句，J2CJ 会将其转换为仓颉的 `for-in` 表达式，并且生成适当的 `iterator()` 方法。

7.8.5 `break` 语句

不支持 Java 的带标签 `break` 语句，因为没有合适的方式将它们转换至仓颉。J2CJ 将它们转换成标记。

7.8.6 `continue` 语句

不支持 Java 的带标签 `continue` 语句，因为没有合适的方式将它们转换至仓颉。J2CJ 将它们转换成标记。

7.8.7 `synchronized` 语句

J2CJ 通过为对象添加监视器的方式支持 `synchronized` 语义。由于字符串和数组类型的参数采用按值传递方式，不支持 `synchronized` 语义。

7.8.8 `try` 语句

不支持 `try-with-resources`。J2CJ 会将这样的语句转换成普通的 `try` 语句，部分内部映射缺失并生成标记。

7.8.9 模式匹配

不支持模式匹配。

7.9 表达式

7.9.1 主要表达式

7.9.1.1 this

J2CJ 能够转换 Java 代码中的构造函数内调用实例方法及传递 **this** 参数的场景，但需要注意的是，转换后得到的仓颉代码不具备可编译性。

理由：

在 Java 构造函数中，调用实例方法并传递 **this** 参数是可行的，但并不推荐，因为对象仍在构造中。之所以可以这样做，是因为在 Java 规范下，新创建的对象在构造函数开始调用前，其字段会自动初始化为默认值。而在仓颉中，只有在类不是 **open** 并且编译器能够保证在调用实例方法之前所有字段都已初始化的情况下，才允许在构造函数中调用实例方法。

```
class Foo{
    var x: Int32
    init() {
        this.bar()      // 错误: 'x' 还未初始化
        x = 0
        this.bar()      // OK
    }
    func bar(): Int32 {
        return x
    }
}
```

虽然 J2CJ 会生成显式的字段初始化器以匹配 Java 默认值的语义，但在实际应用中，许多 Java 类并不被声明为 **final**，因此它们会在 J2CJ 输出中声明为 **open**，导致编译失败。

如果在仓颉编译器的日志中看到 `instance member function 'bar' cannot be accessed in the constructor of open class 'Foo'` 或 `'this' is not allowed to be accessed before all member variables are initialized` 等类似错误消息，请检查原始 Java 源代码中是否有这样的 **this** 使用，并相应地进行重构。

7.9.2 this 表达式

不支持形如 `_类型名_.this` 的合格 **this** 表达式，这种表达式可能出现在 Java 内部类中。

7.9.3 字段访问表达式

在 Java 中，声明为静态的字段仍然可以通过实例访问：

```
interface WithAField {
    String FIELD = "field";    // 隐式地是 public final static
    . . .
}

static int foo(WithAField i) {
    String s1 = i.FIELD;
    String t2 = getWAF().FIELD; // getWAF() 返回一个 WithAField
    . . .
}
```

但在 Cangjie 中，这是不可能的。

如果在字段访问表达式中出现在点 `.` 左侧的表达式没有副作用，并且字段是静态的，通常可以将其替换为相应的类型名称。当前版本的 J2CJ 确实会进行这种替换，但仅限于接口常量，并且前提是该表达式是一个简单的标识符。在所有其他情况下，替换不会发生，生成的 Cangjie 代码无法编译，需要手动检查。

在默认的“代码风格优先”模式（`-mode=codestyle`）下，即没有 `Option<T>` 包装的情况下，上述 Java 代码将转换为：

```
interface WithAField {
    static prop FIELD: String {
        get() {
            "field"
        }
    }
    . . .
}

static func foo(WithAField i) {
    let s1 = WithAField.FIELD; // 'i' 现在可能未被使用
    let s2 = getWAF().FIELD;  // 无法编译
    . . .
}
```

注意：Cangjie 接口可能根本不能包含字段，因此 Java 接口常量在可能的情况下会被转换为只读的静态属性（详见字段（常量）声明）。

7.9.4 方法调用表达式

不支持使用限定 `super` 形式的方法调用表达式：

`TypeName . super . [ TypeArguments ] Identifier`

无论是在内部类中还是在接口中都不支持。

7.9.5 方法引用表达式

支持方法引用表达式，但存在以下限制：

- 不支持法引用表达式的类型为**交集类型**。
- 不支持**实例初始化器**和**静态初始化器**中出现的方法引用表达式。

实例初始化器和静态初始化器代码被转化时，J2CJ 会跳过方法引用表达式，直接将转化前源码复制到输出文件中，并进行**标记**。

以下相关功能现已得到支持：

- 在默认的“代码风格优先”模式（`-mode=codestyle`）下，`java.util.concurrent.Callable` 和 `java.util.function` 包中的接口现在被映射为函数类型。它们的默认方法，如 `andThen()`，仍然不受支持。

以下这些功能尚未实现：

- 流（`java.util.stream`）

7.9.6 关系运算符

7.9.6.1 类型比较运算符 `instanceof`

由于仓颉中没有 `null`，所以引用类型的值被包裹在 `Option<T>` 中，Java 的 `instanceof` 运算符可能无法直接转换为仓颉运算符。可使用泛型 helper 类 `utils.ValueBox<T>` 来进行类型比较。

不支持数组类型的比较，因为在仓颉中，数组不是对象而是结构体。另外，仓颉的数组是不可变的，而 Java 的数组是协变的，即如果 `B` 是 `A` 的子类型，则 `B[]` 是 `A[]` 的子类型。因此，在仓颉中，数组类型的比较是没有意义的。

7.9.7 lambda 表达式

Java lambda 表达式部分支持，但只有其中一部分会被翻译成仓颉 lambda 表达式，而且这种情况仅发生在默认的“代码风格优先”模式（`-mode=codestyle`）下。

J2CJ 支持以下 lambda 相关特性：

- 使用在 lambda 表达式外部声明的 `final`/有效 `final` 变量
- 方法引用
- 嵌套 lambda 表达式
- 在 lambda 表达式体内声明局部类
- 在默认的“代码风格优先”模式下（`-mode=codestyle`）。`java.util.concurrent.Callable` 和接口 `java.util.function` 包中的函数现在被映射为函数类型，但它们的默认方法，如 `andThen()`，仍然不被支持。

以下功能尚未实现：

- 流（`java.util.stream`）

不支持：

- 交集类型
- 实例初始化和静态初始化器中的 lambda 表达式（转换时会被包装在标记中）

由于 Java 和仓颉中 lambda 表达式的性质差异过大，直接将 Java 的 lambda 表达式转换为仓颉表达式在一般情况下是不可行的。为了解决这些差异，转换器首先会将 Java 中的 lambda 表达式转换为匿名类，然后再将这些类转换为仓颉代码：

```
@FunctionalInterface
interface IntegerFunction {
    int f(int x);
}

public class Simple {
    static int apply(IntegerFunction f, int x) {
        return f.f(x);
    }
    public static void main(String[] args) {
        apply((x) -> x + 1, 0);
    }
}
```

```
// Non-essential boilerplate removed for clarity
interface IntegerFunction {
```

```

    func f(x: Int32): Int32
  }

  // Synthetic class representing the lambda
  open class Simple_1 <: IntegerFunction {
    override public open func f(x: Int32): Int32 {
      return x + 1i32
    }
  }

  public open class Simple {
    static func apply(f: ?IntegerFunction, x: Int32): Int32 {
      return f.getOrThrow().f(x)
    }
    public static func javaMain(args: ?Array<?String>): Unit {
      apply(Simple_1(), 0i32)
    }
  }
}

```

在默认的“代码风格优先”模式（`-mode=codestyle`）中，J2CJ 现在可以将 Java 函数式接口类型的 `_uses_` 转换为仓颌函数类型，以及将实现类的实例转化为成员的使用适当时的函数引用（有关详细信息，请参阅功能接口。从而实现了将 Java lambda 表达式转换为仓颌 lambdas

上面的示例现在将转换为：

```

// Non-essential boilerplate removed for clarity
interface IntegerFunction {
  func f(x: Int32): Int32
}

public class Simple {
  // Notice the use of a function type instead of IntegerFunction:
  static func apply(f: (Int32) -> Int32, x: Int32): Int32 {
    return f(x)
  }
  public static func javaMain(args: Array<String>): Unit {
    // Java lambda translated into a Cangjie lambda:
    apply({ x => x + 1 }, 0i32)
  }
}

```

有关详细信息，请参阅[函数类型](#)。

7.10 其他

7.10.1 异常处理

没有针对 `Throwable.initCause()`、`Throwable.getCause()` 和相应的 `Throwable` 构造函数的映射。如果尝试以另一个异常作为 `cause` 来实例化新的 `Throwable` 对象，则会被转换为调用无参数构造函数和标记。

7.10.2 元类型编程

当前只支持类字面量以及 `Object.getClass()`、`Class.getName()`、`Class.getName()`、`Class.getResourcesAsStream()` 方法。不支持反射 API (`java.lang.reflection`)。

8 标记

8.1 No type mapping: `<CLASS_FQ_NAME>`

说明：没有针对 Java 类完全限定名`<CLASS_FQ_NAME>`的映射。

解决办法：如果在仓颉标准库或用户自己的仓颉代码库中有适合`CLASS_FQ_NAME`的简单映射，可以尝试添加该映射。

否则，从以下任选一项：

- 自行实现一个仓颉等效类型，并将`<CLASS_FQ_NAME>`映射到该类型。
- 重构 Java 源代码，使其不再使用该类。

具体选择哪种方案取决于该类的复杂度及其在代码库中使用的广泛程度，用户可以根据实际情况做出决策。

8.2 Missing mapping for `<CLASS_FQ_NAME>` member/constructor: `<MEMBER_NAME>`

说明：没有针对`<CLASS_FQ_NAME>.<MEMBER_NAME>`成员或构造函数（具有与提供的实参相同数量和类型的参数）的映射。这可能意味着该命名成员完全没有映射，或者现有的映射无法应用于给定的实参集。

解决办法：如果仓颉中有合适的简单映射，可以尝试在`mappings.cjmap`或者其他`.cjmap`文件中添加该映射。具体操作请参见本文档中的“mappings”部分。

8.3 Missing method of mapped type `<CLASS_FQ_NAME>`: `<MEMBER_NAME>`

说明：成员所有者`<CLASS_FQ_NAME>`已成功映射，但没有针对给定实参集的`<MEMBER_NAME>`的正确映射。

解决办法：如果仓颉中有合适的简单映射，可以尝试在`mappings.cjmap`或者其他`.cjmap`中添加该映射。具体操作请参见本文档中的“mappings”部分。

8.4 Mapping is present but failed

说明：存在失败映射的原因：无法强制转换实参、无法强制转换 receiver、内部错误等。

解决办法：请上报 issue 至社区，需要工具侧修改。

8.5 Unimplemented feature `<FEATURE_NAME>`

说明：`<FEATURE_NAME>`功能的转换尚未在 J2CJ 中实现。

解决办法：重写 Java 源代码，使其不再使用`<FEATURE_NAME>`。

8.6 Unsupported `<FEATURE_DESCRIPTION>`

说明：具有`<FEATURE_DESCRIPTION>`描述的功能/场景未在 J2CJ 中实现。

解决办法：重写 Java 源代码，使其不再使用`<FEATURE_DESCRIPTION>`。

8.7 No type transform for: <EXPRESSION> <FROM_TYPE> -> <TO_TYPE>

说明：在转换<EXPRESSION>时，J2CJ 期望得到<TO_TYPE>，但实际上得到了<FROM_TYPE>。

解决办法：此标记常与“No ... mapping”标记共同出现。尝试先解决“No ... mapping”标记。

8.8 Cannot infer type argument: <TYPE_PARAMETER>

说明：J2CJ 无法将一个泛型 Java 类型、方法或构造函数转换为带参数的仓颉成员，因为无法推断出给定的<TYPE_PARAMETER>的类型实参。

这可能是由于缺少映射导致的误报，或者是当前 J2CJ 架构的限制。参考以下例子：

```
class TypeParameterLostInTranslation {
    public Foo test() {
        return instantiate(Foo.class);
    }

    private <T> T instantiate(Class<T> clazz) {
        return null;
    }
}

class Foo {}
```

在 Java 中，类字面量的类型 `java.lang.Class` 是泛型，而仓颉中的对应类型 `std.reflect.TypeInfo` 则不是。当前的 J2CJ 还无法妥善处理此类情况：

```
open class TypeParameterLostInTranslation <: Hashable & ToString {
    init() { }

    public open func test(): ?Foo {
        return castOrThrow<T, Foo>(instantiate<-- Cannot infer type argument: T
        -->>(TypeInfo.of<Foo>() as TypeInfo))
    }

    private func instantiate<T>(clazz: ?TypeInfo): ?T {
        return None
    }
}
```

解决办法：这种错误通常是由缺少某个相关 Java 实体映射导致的，检查周围是否有“No ... mapping”标记。如果有，先解决这些标记，然后重新运行 J2CJ。

如果仍然报错，需要手动编辑 J2CJ 输出，同时请上报 issue 至社区。

8.9 Generic instantiation failure: <CALL>

说明：J2CJ 无法收集<CALL>调用的类型实参。

解决办法：此标记常与“No ... mapping”标记共同出现。尝试先解决“No ... mapping”标记错误。

8.10 Constraints not satisfied <FAILURES>: <CALL>

说明：J2CJ 无法获取<CALL>调用的类型实参。

解决办法: 手动编辑 J2CJ 输出, 同时请上报 issue 至社区。

8.11 Expression is too deep: <EXPRESSION>

说明: <EXPRESSION>表达式包含过多嵌套调用: `a(b(c...))`

解决办法: 在 Java 代码中, 将内层调用移到表达式外部: `a(b(c(d()))) => var tmp = c(d()); a(b(tmp))`
另一种解决方案是使用 `max.expression.depth` 属性 (默认为 5) 来更改最大表达式深度。注意: 增加最大表达式深度可能会导致转换过程执行时间过长。

8.12 Constants in interfaces with bounded type parameters are unsupported: <INITIALIZER>

说明: Cangjie 接口中不能包含常量, 因此它们会被转换为静态只读属性。如果初始化表达式比单个字面量更复杂, 则需要生成一些辅助代码。当前版本的 J2CJ 只能为没有类型参数约束的接口生成此类辅助代码。

解决办法: 有两种选择:

1. 如果初始化表达式可以在编译前完全求值, 则将其替换为单个字面量。
2. 如果初始化表达式可以在接口外部求值, 则手动编辑生成的代码。

考虑以下示例:

```
interface I<T extends Comparable> {
    String FIZZBUZZ = "Fizz" + "Buzz";
}
```

```
private let I_FIZZBUZZ = <-- Constants in interfaces with bounded type parameters
are unsupported: I3 -->.FIZZBUZZ_init
interface I<T> where T <: Comparable<Any> {
    static prop FIZZBUZZ: String {
        get() { I_FIZZBUZZ }
    }

    static prop FIZZBUZZ_init: String {
        get() { "Fizz" + "Buzz" }
    }
}
```

第一种选择是在翻译之前, 将初始化器替换为 Java 代码中的字面量:

```
interface I<T extends Comparable> {
    String FIZZBUZZ = "FizzBuzz";
}
```

```
interface I<T> where T <: Comparable<Any> {
    static prop FIZZBUZZ: String {
        get() { "FizzBuzz" }
    }
}
```

对于第二种选择, 很明显在这种情况下不需要使用第二个 `_init` 属性的初始化技巧, 因此代码可以重写为:

```
private let I_FIZZBUZZ = "Fizz" + "Buzz"
interface I<T> where T <: Comparable<Any> {
    static prop FIZZBUZZ: String {
        get() { I_FIZZBUZZ }
    }
}
```

```
}  
}
```

如果以上两种选择都不适用，则可能需要更复杂的重写方式。

有关详细信息，请参见 [字段（常量）声明](#)。

8.13 Unary operators usage with volatile fields is not supported: <EXPRESSION>

说明：目前还不支持对 **volatile** 字段使用一元运算符和复合赋值运算符。有关详细信息，请参阅 [volatile 字段](#)。

解决方法：要么重写原始的 <EXPRESSION>，避免使用一元运算符或对 **volatile** 字段使用复合赋值运算符，然后重新运行 J2CJ，要么手动将该表达式进行转换。

8.14 Compound assignment to volatile fields is not supported: <EXPRESSION>

说明：目前还不支持对 **volatile** 字段使用复合赋值运算符和一元运算符。有关详细信息，请参阅 [volatile 字段](#)。

解决方法：要么重写原始的 <EXPRESSION>，不使用复合赋值运算符对 **volatile** 字段进行操作，并重新运行 J2CJ，要么手动将该表达式进行转换。

8.15 Internal error

说明：J2CJ 内部错误。可能是由于提供了错误的 Java 源代码或不完整项目作为输入导致的。

解决办法：检查你的 Java 项目是否可以成功构建。如果构建失败，请先解决导致失败的问题，然后重新运行 J2CJ。

否则，这可能是一个无法从用户端修复的真正 bug。请提交一份报告，附上 J2CJ 日志以及一个足以重现错误的代码示例。